

Open Data Sharing in the Lakehouse Era



Zac Davies

ANZ Field CTO, Databricks
Sydney Data Engineering Meetup

Talking points

What I'll ~~delta~~ share with you

- Overview of Delta Sharing
- Common patterns
- Deep dive:
Examples, Performance, Interoperability
- Q&A



Evolution of data sharing



Evolution of data sharing



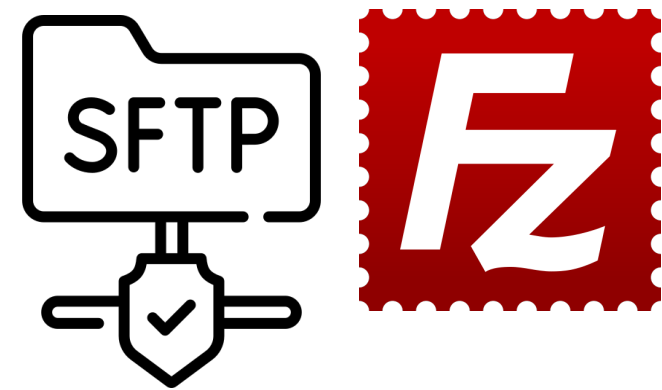
Evolution of data sharing



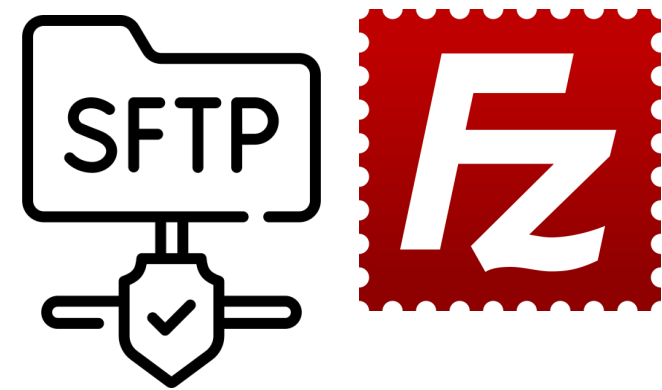
Evolution of data sharing



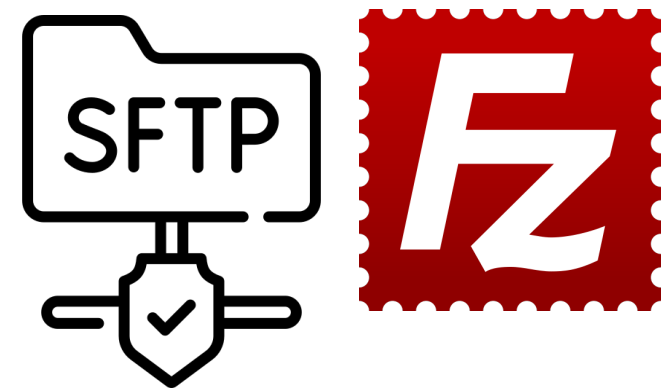
Evolution of data sharing



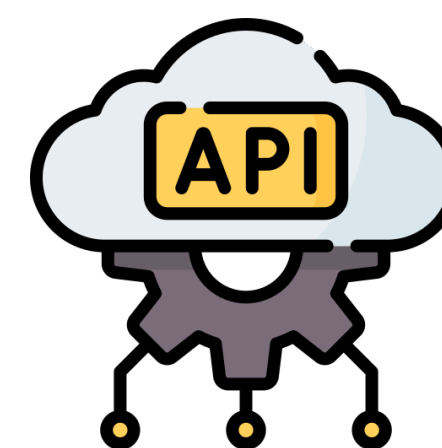
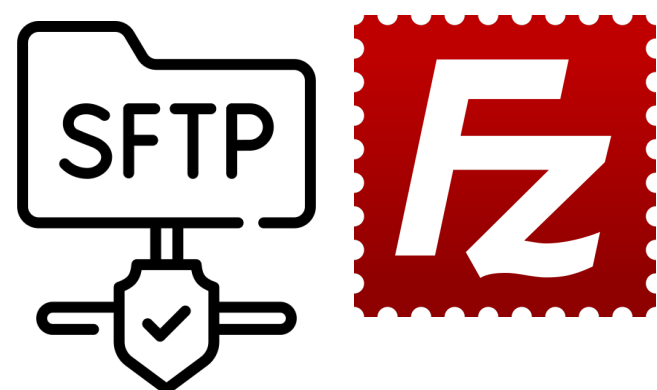
Evolution of data sharing



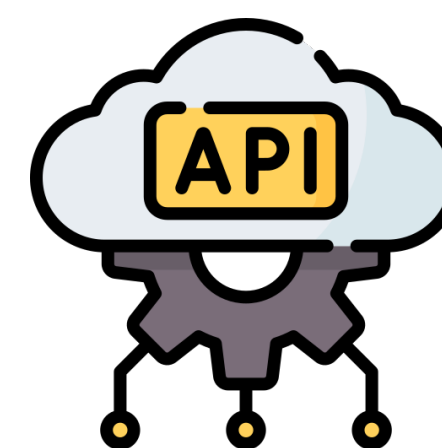
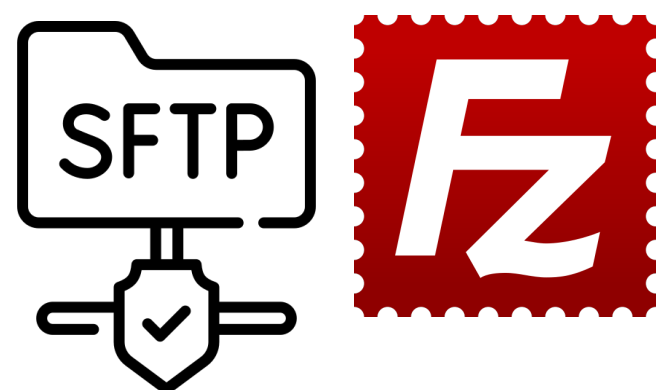
Evolution of data sharing



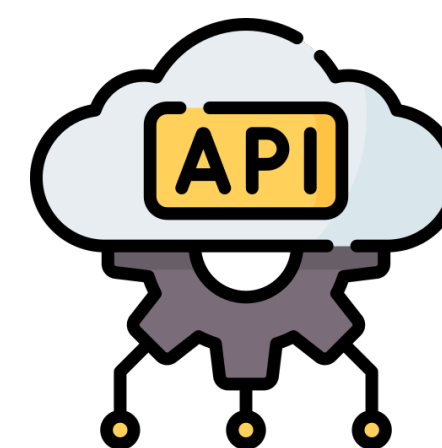
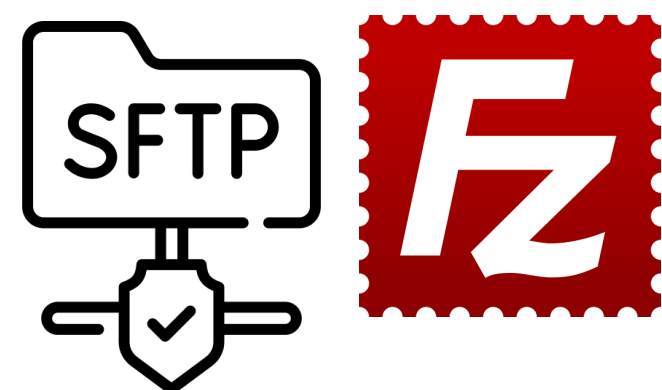
Evolution of data sharing



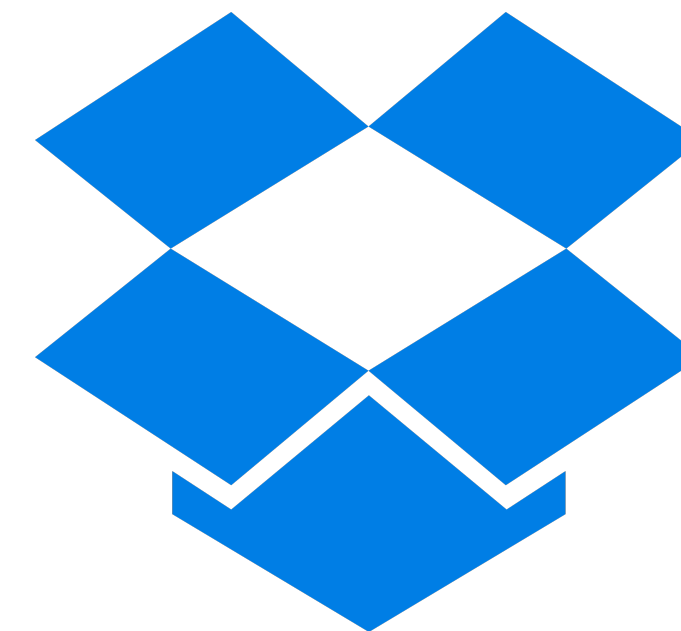
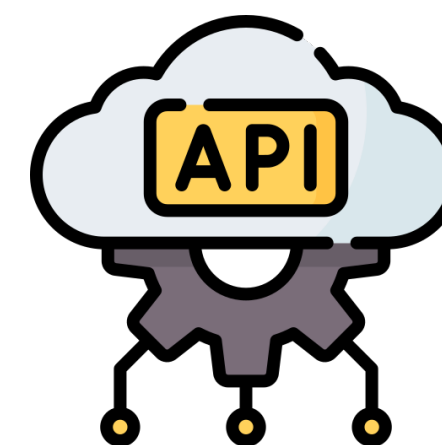
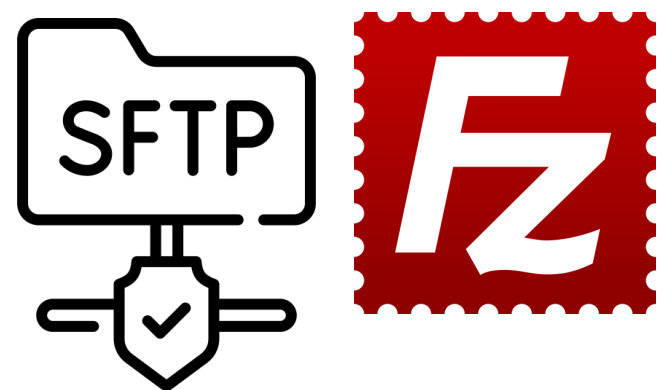
Evolution of data sharing



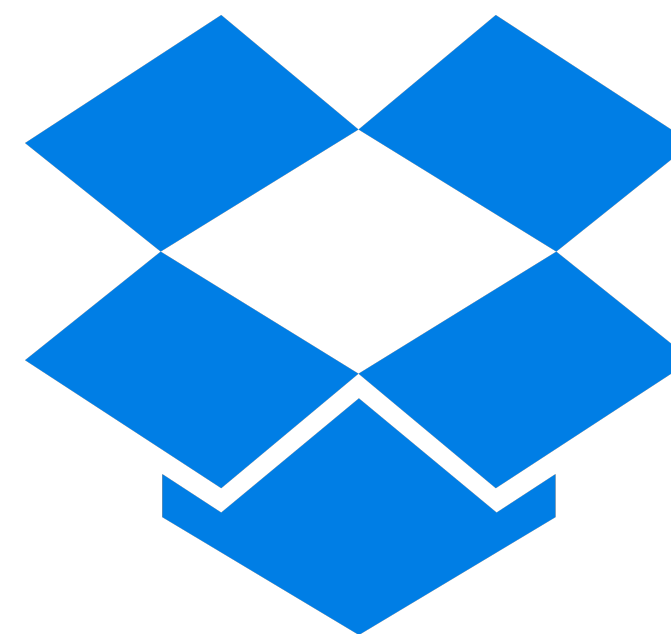
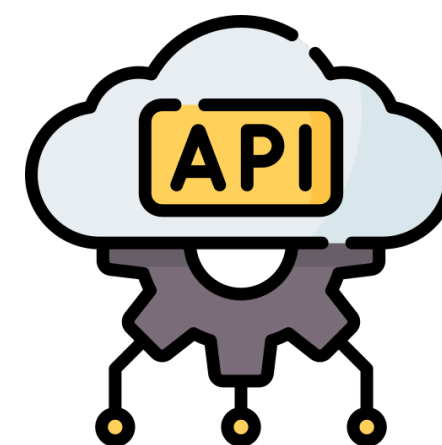
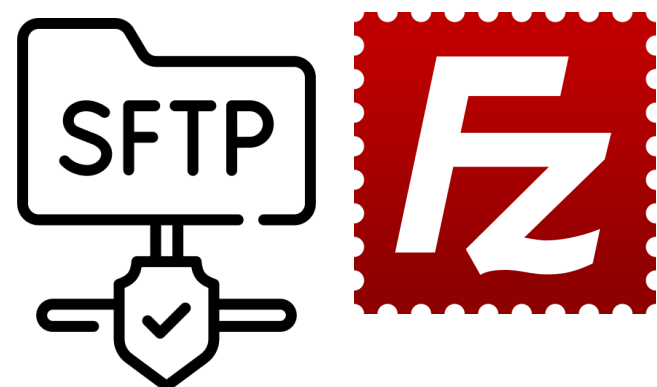
Evolution of data sharing



Evolution of data sharing

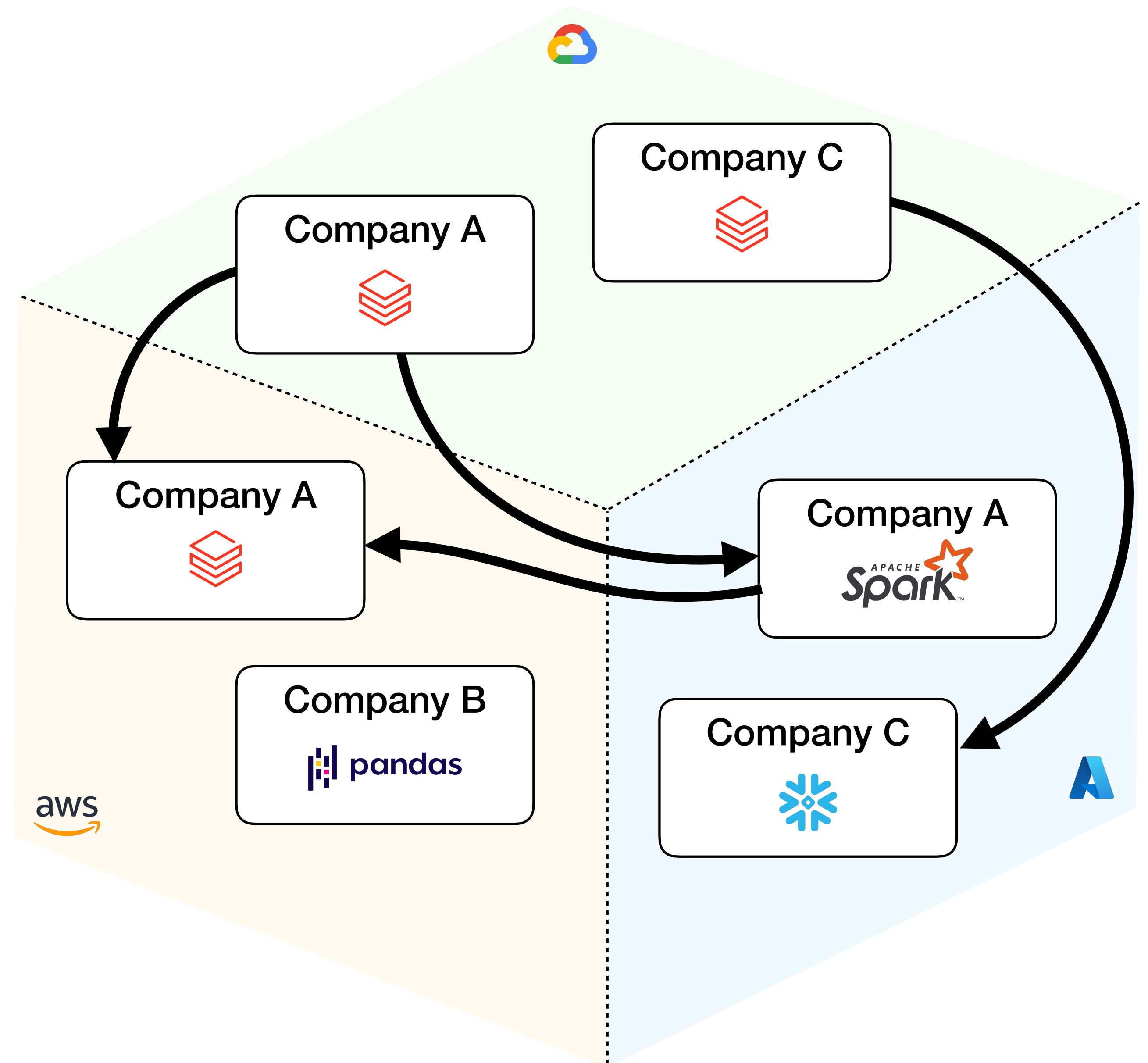


Evolution of data sharing



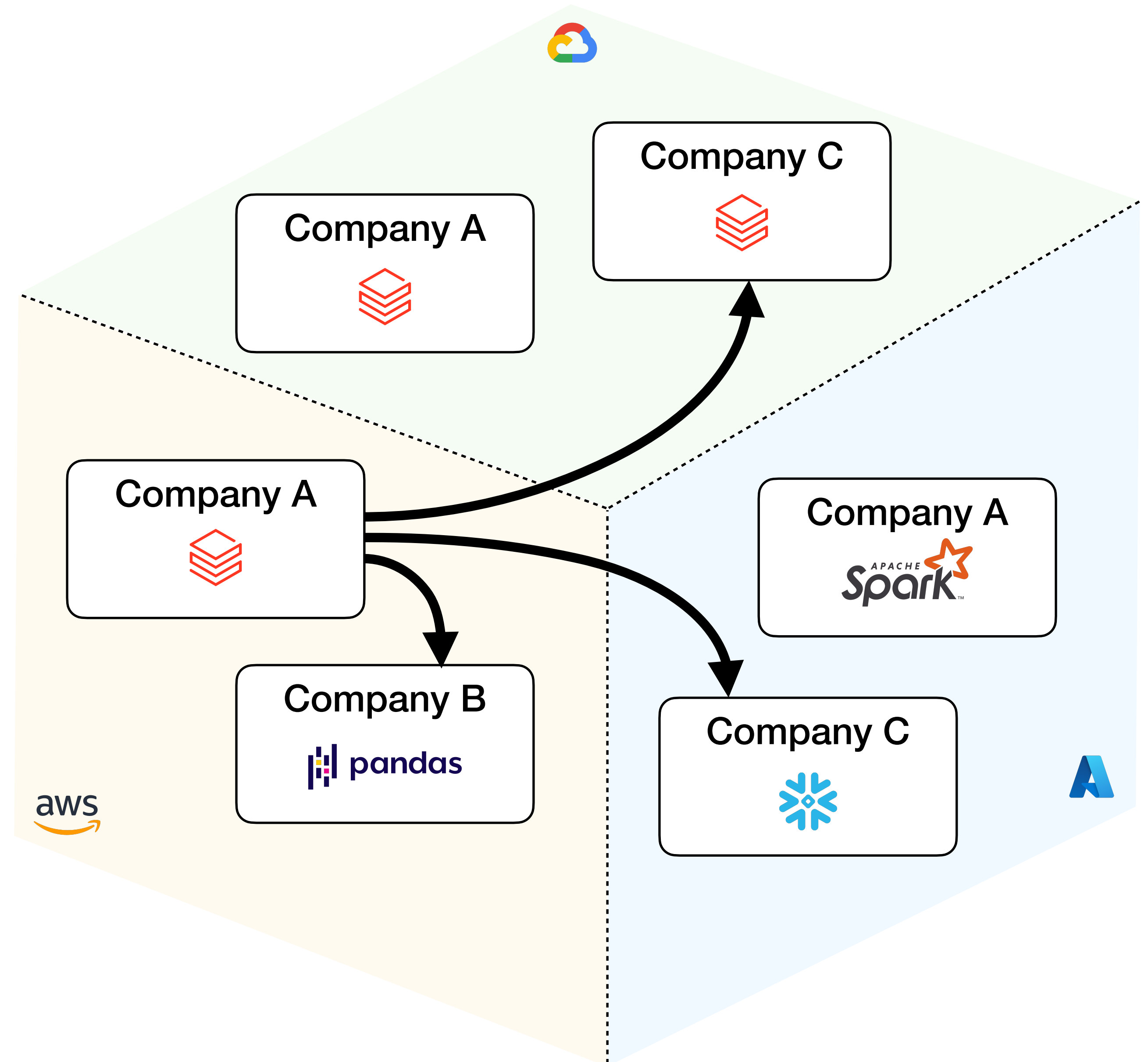
Example Use Cases

- Sharing within organisations (e.g. x-region, x-department)
- Between partners (B2B) (e.g. data partnerships)
- Open exchanges / marketplaces (e.g. Covid-19 datasets)



Example Use Cases

- Sharing within organisations (e.g. x-region, x-department)
- Between partners (B2B) (e.g. data partnerships)
- Open exchanges / marketplaces (e.g. Covid-19 datasets)



High level overview

Zero-copy data access



High level overview

Zero-copy data access

Data Provider

Table

Col1	Col2	Col3	...
X	1	{}	...
X	2	{1,2}	...
Y	3	{}	...
Y	4	{10}	...



GCS / S3 / ADLS



High level overview

Zero-copy data access

Data Provider

Table

Col1	Col2	Col3	...
X	1	{}	...
X	2	{1,2}	...
Y	3	{}	...
Y	4	{10}	...

GCS / S3 / ADLS



Data Recipient



High level overview

Zero-copy data access

Data Provider

Table

Col1	Col2	Col3	...
X	1	{}	...
X	2	{1,2}	...
Y	3	{}	...
Y	4	{10}	...

GCS / S3 / ADLS



Sharing
Server

Data Recipient



High level overview

Zero-copy data access

Data Provider

Table

Col1	Col2	Col3	...
X	1	{}	...
X	2	{1,2}	...
Y	3	{}	...
Y	4	{10}	...

GCS / S3 / ADLS



Sharing
Server

Data Recipient



High level overview

Zero-copy data access

Data Provider

Table

Col1	Col2	Col3	...
X	1	{}	...
X	2	{1,2}	...
Y	3	{}	...
Y	4	{10}	...

GCS / S3 / ADLS



Sharing
Server

Data Recipient



Sharing
Client



High level overview

Zero-copy data access

Data Provider

Table

Col1	Col2	Col3	...
X	1	{}	...
X	2	{1,2}	...
Y	3	{}	...
Y	4	{10}	...

GCS / S3 / ADLS



Sharing Server

Data Recipient

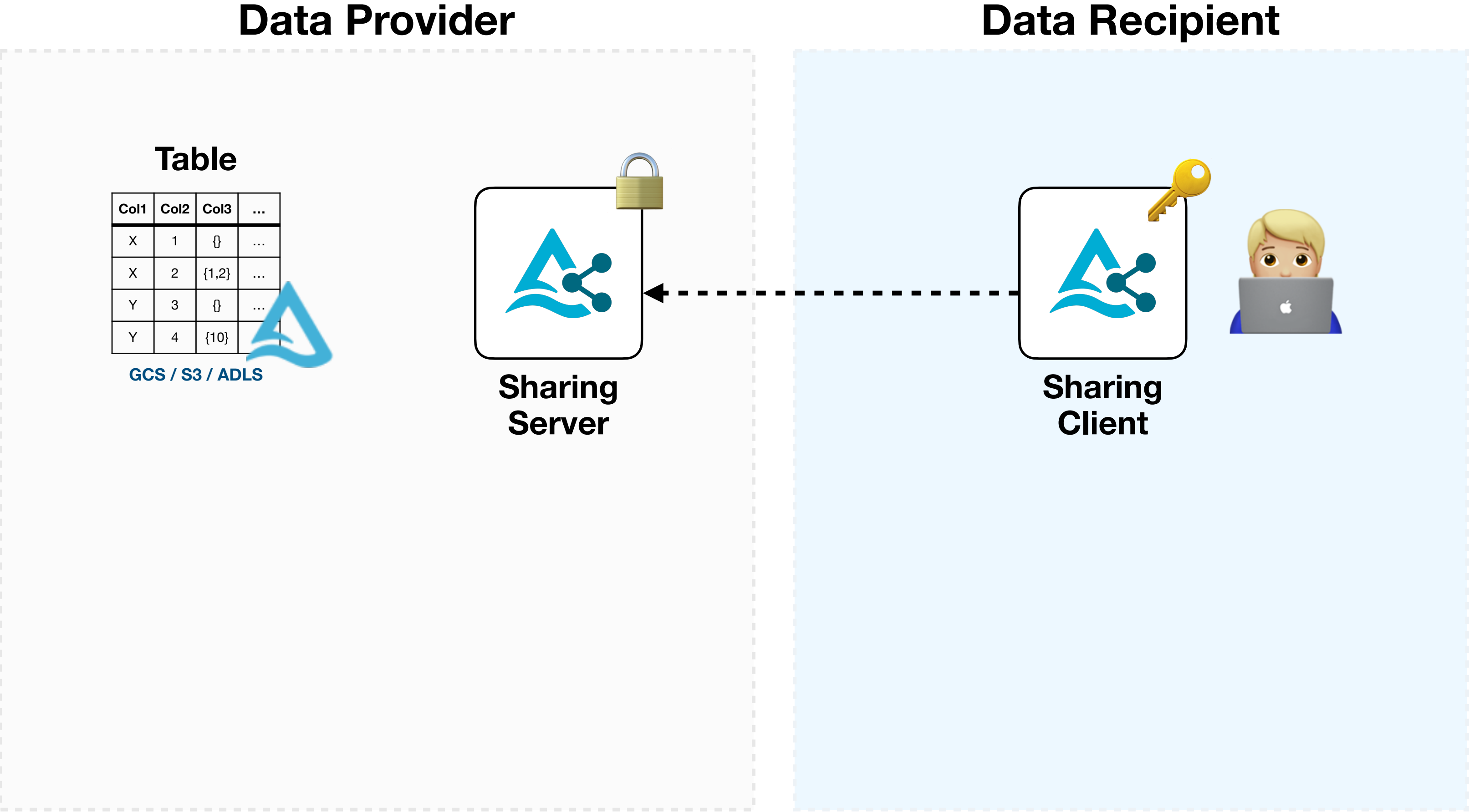


Sharing Client



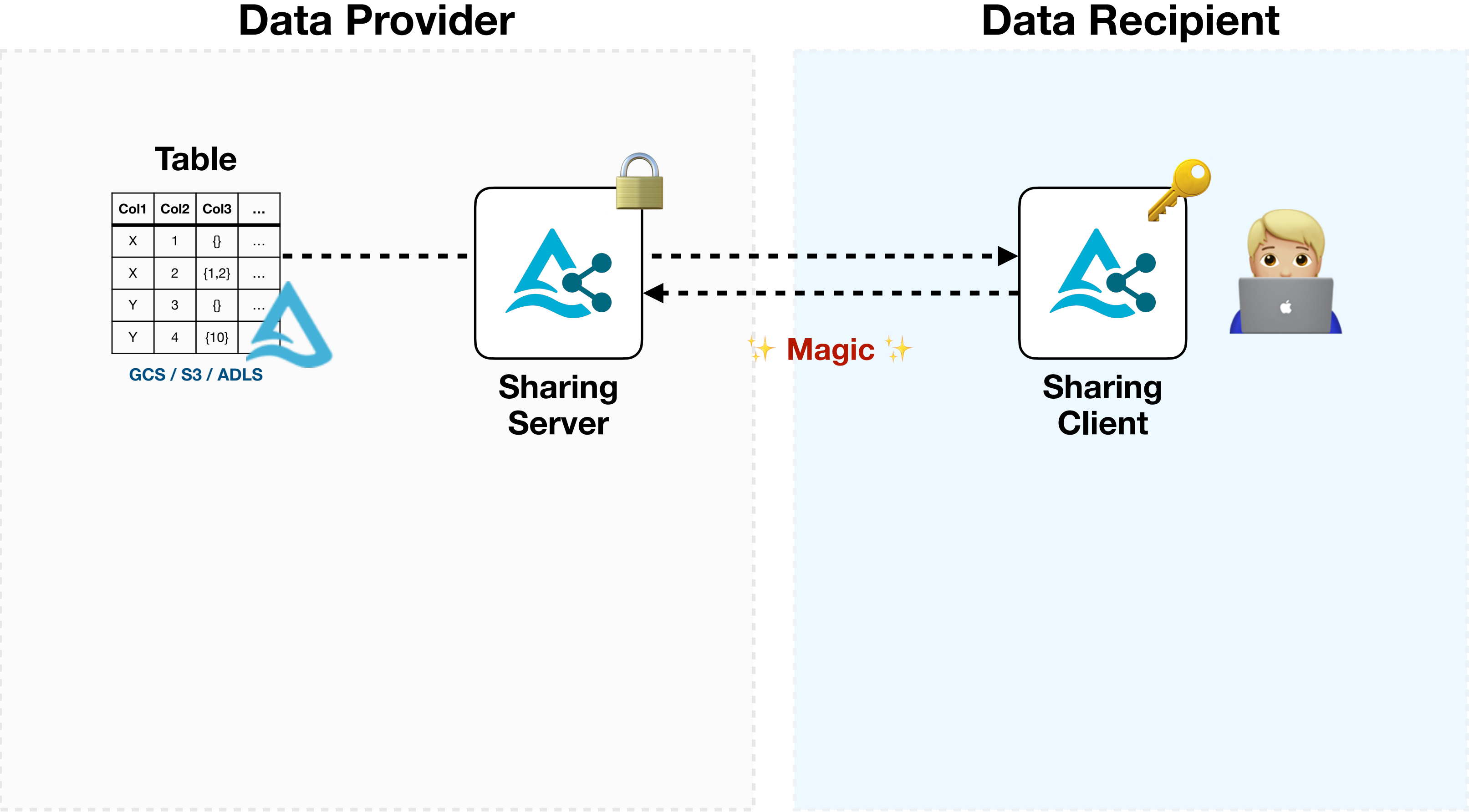
High level overview

Zero-copy data access



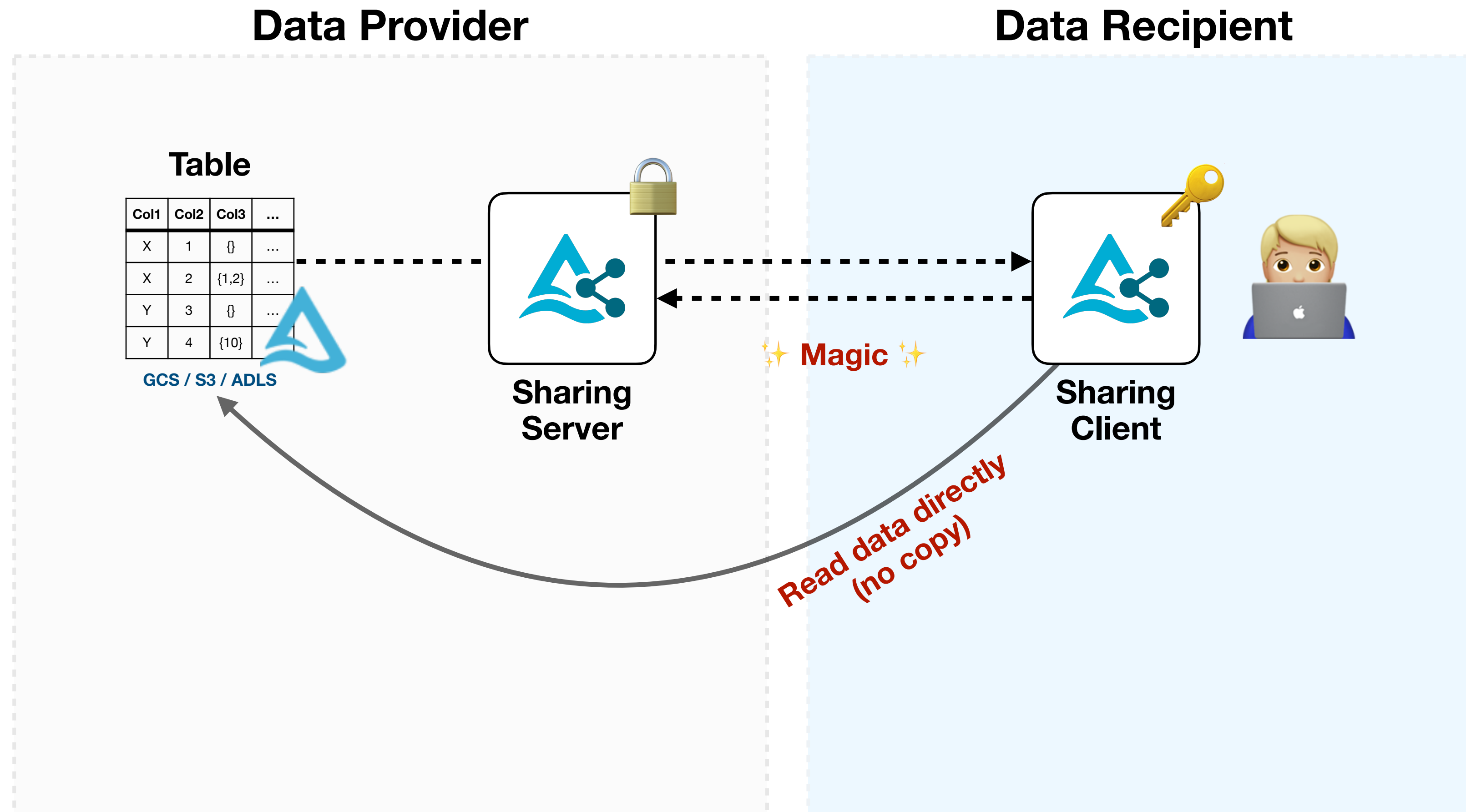
High level overview

Zero-copy data access



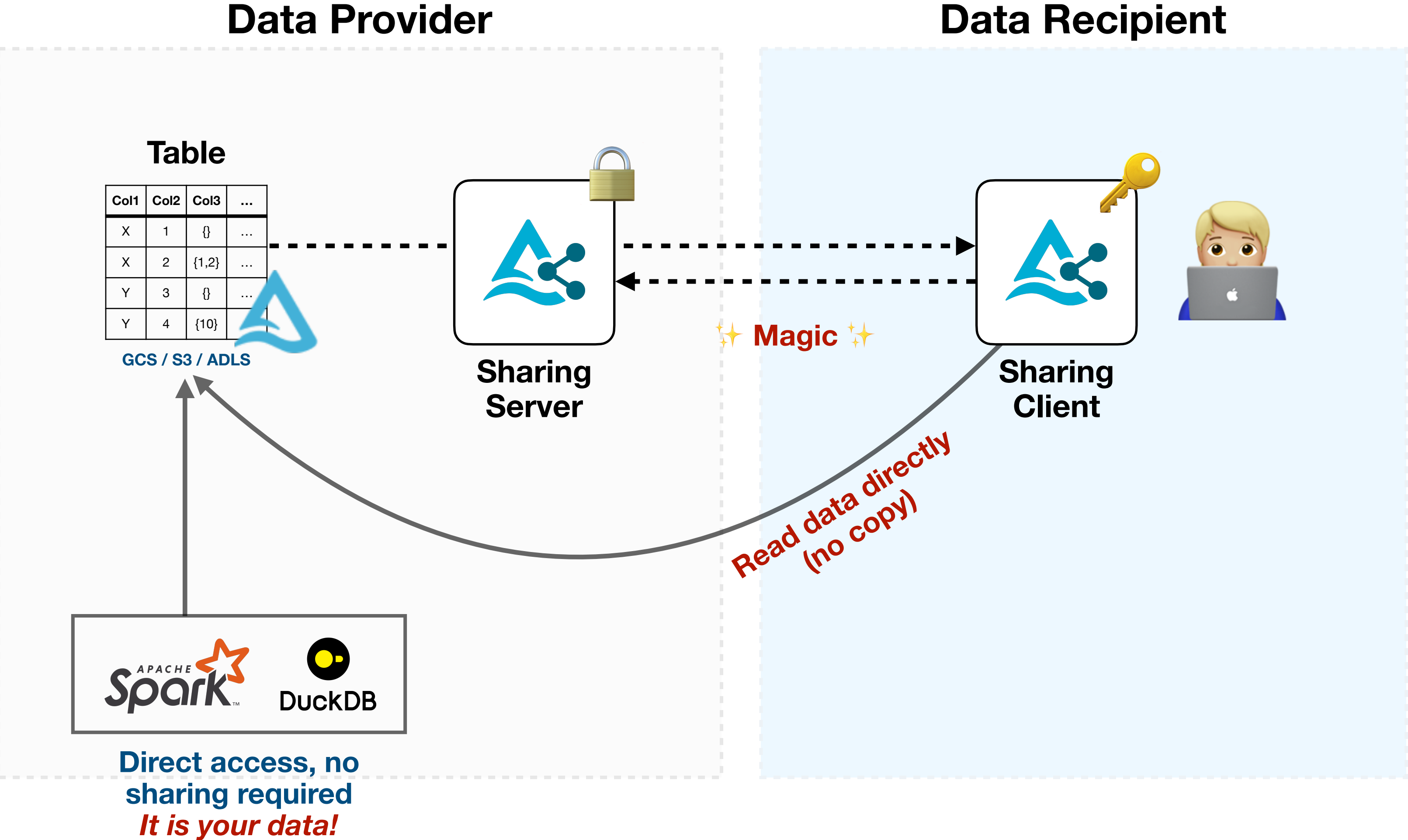
High level overview

Zero-copy data access



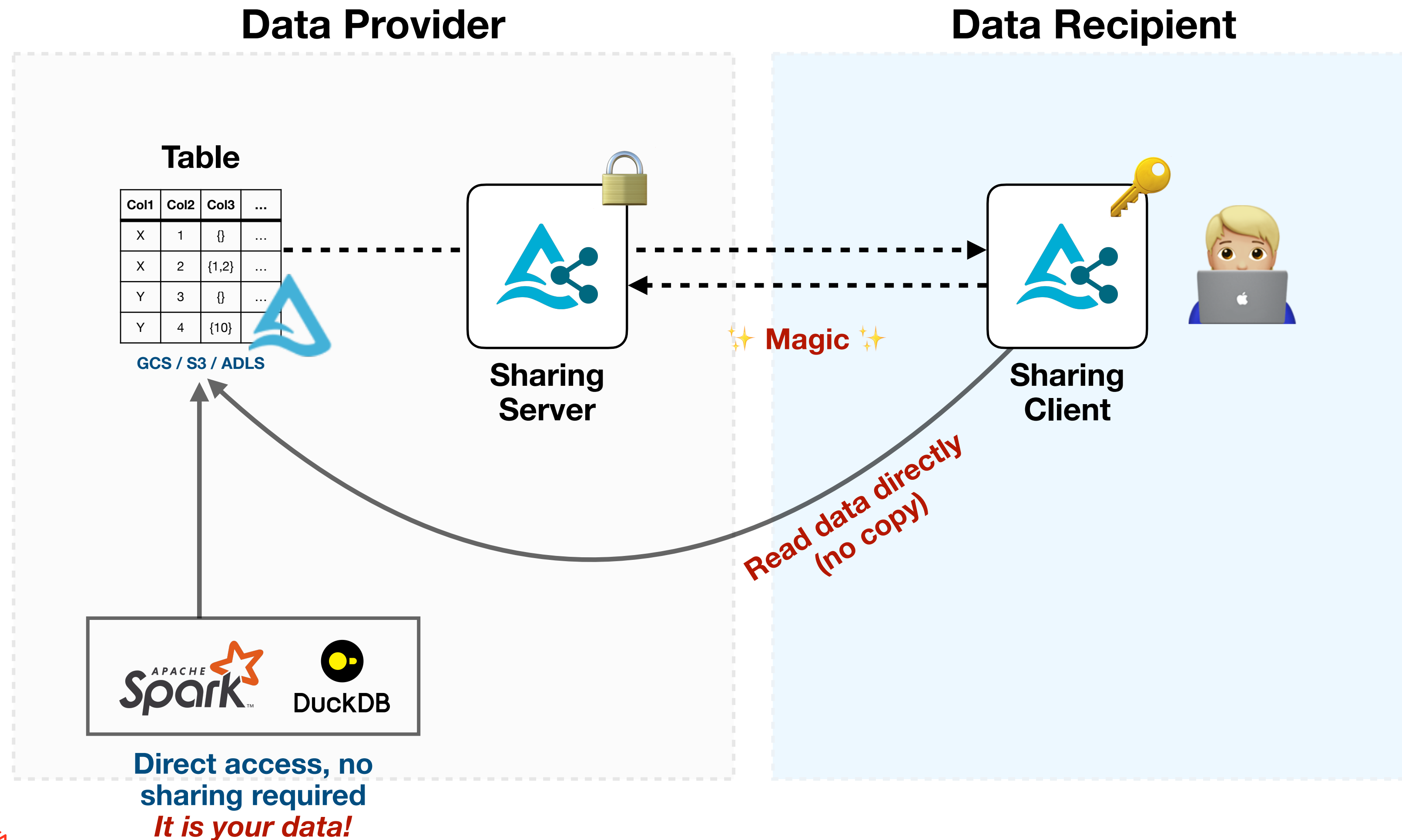
High level overview

Zero-copy data access



High level overview

Zero-copy data access

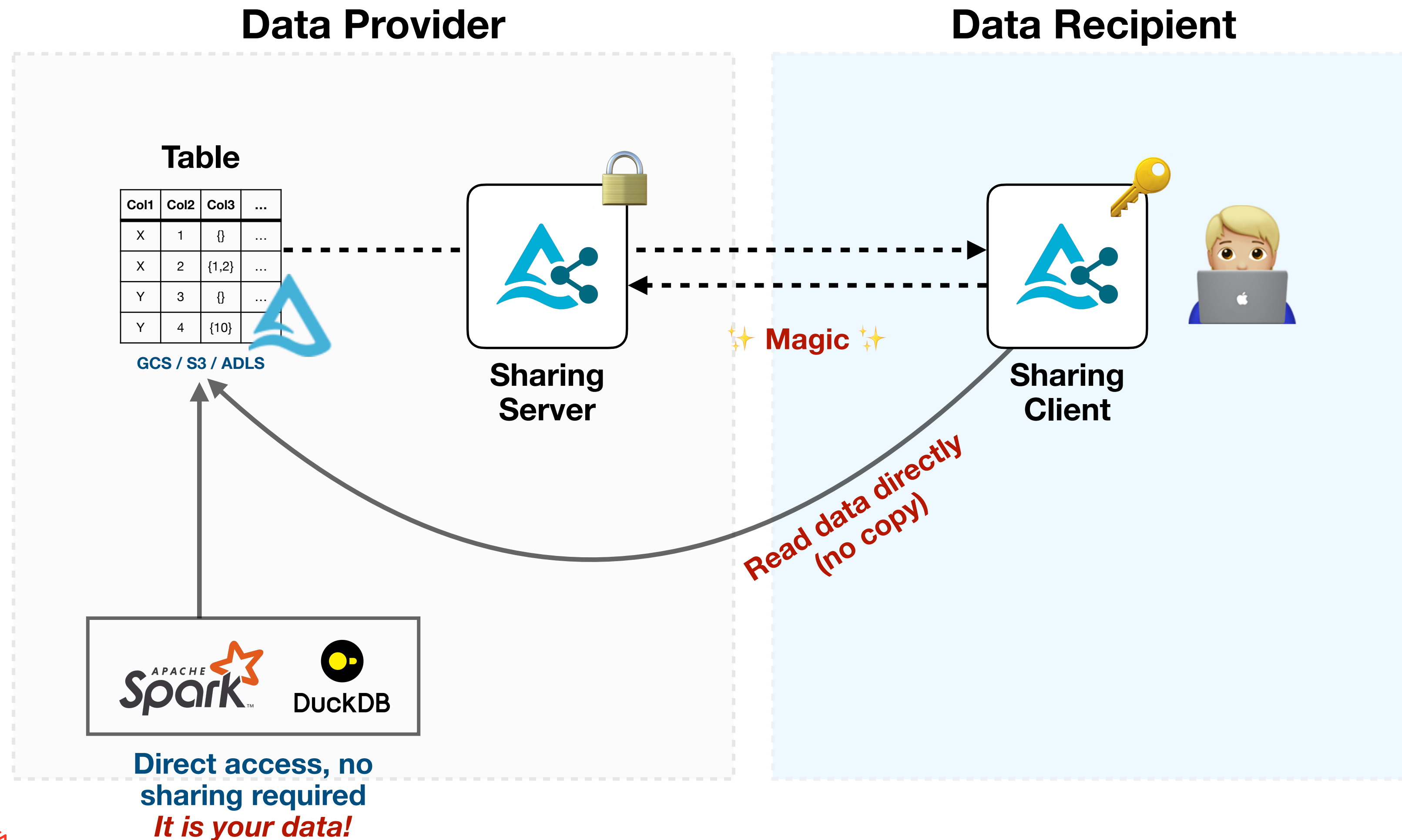


- Provider governs access to tables



High level overview

Zero-copy data access

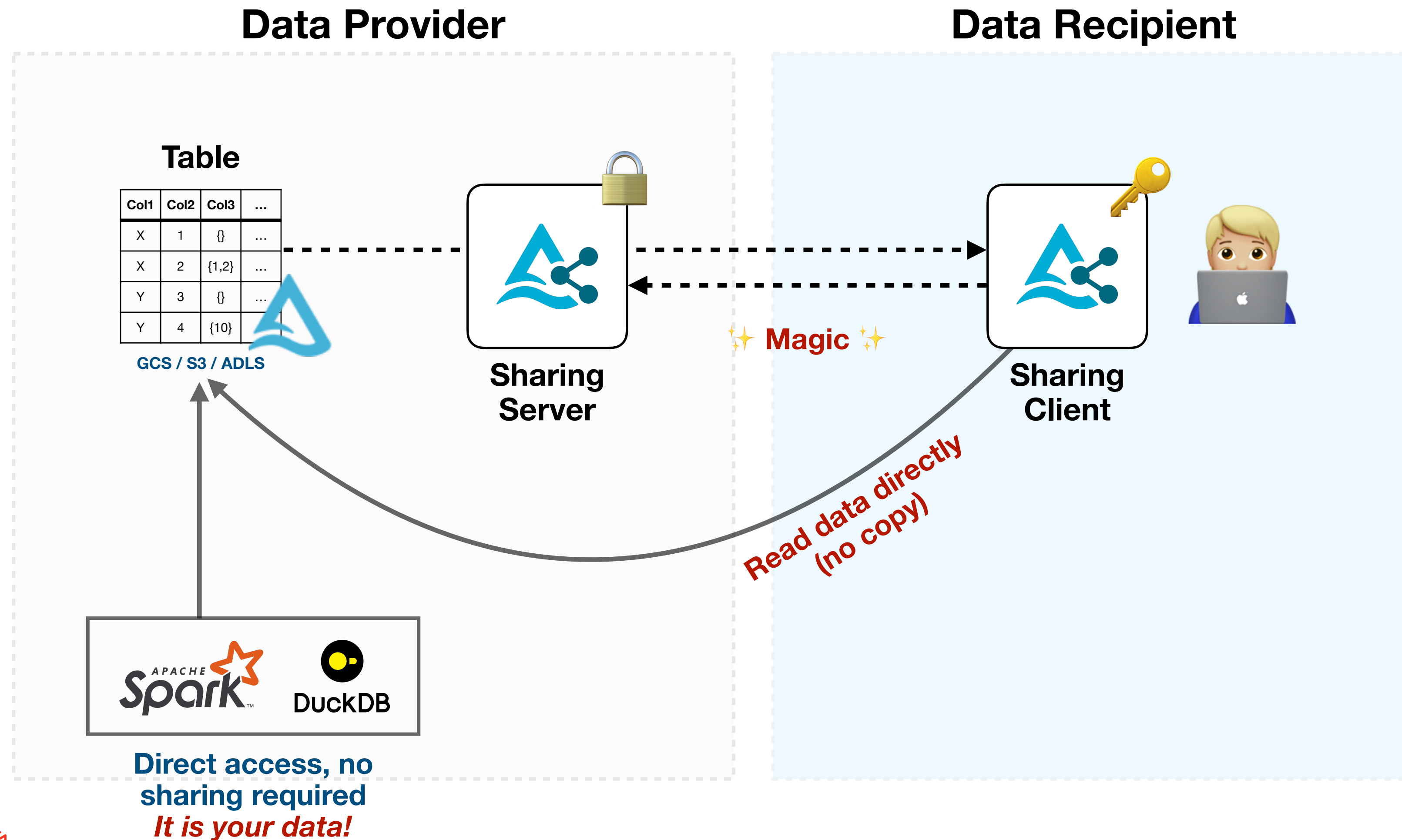


- Provider governs access to tables
- Recipient decides which client they want to use



High level overview

Zero-copy data access

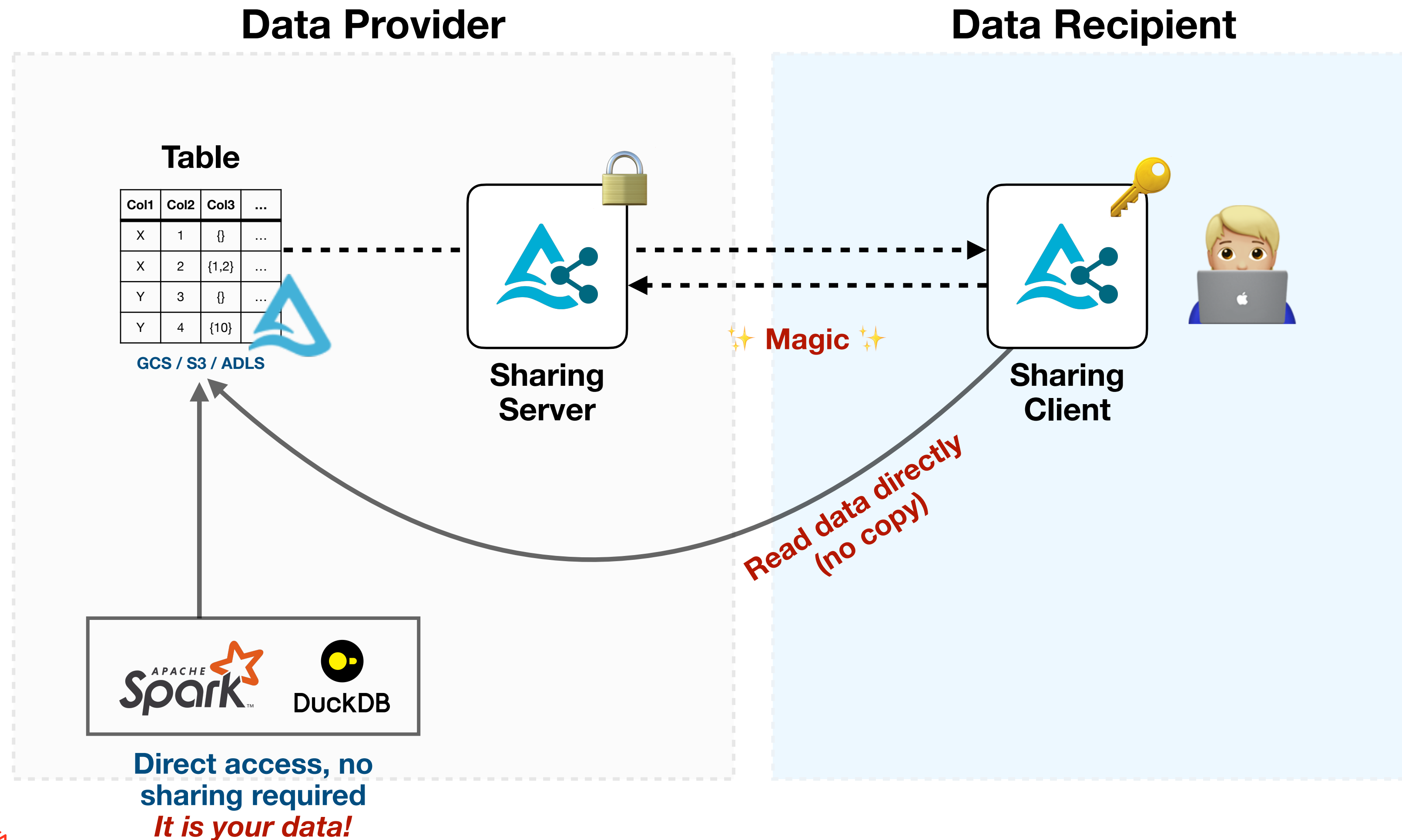


- Provider governs access to tables
- Recipient decides which client they want to use
- Does not change access patterns for provider



High level overview

Zero-copy data access

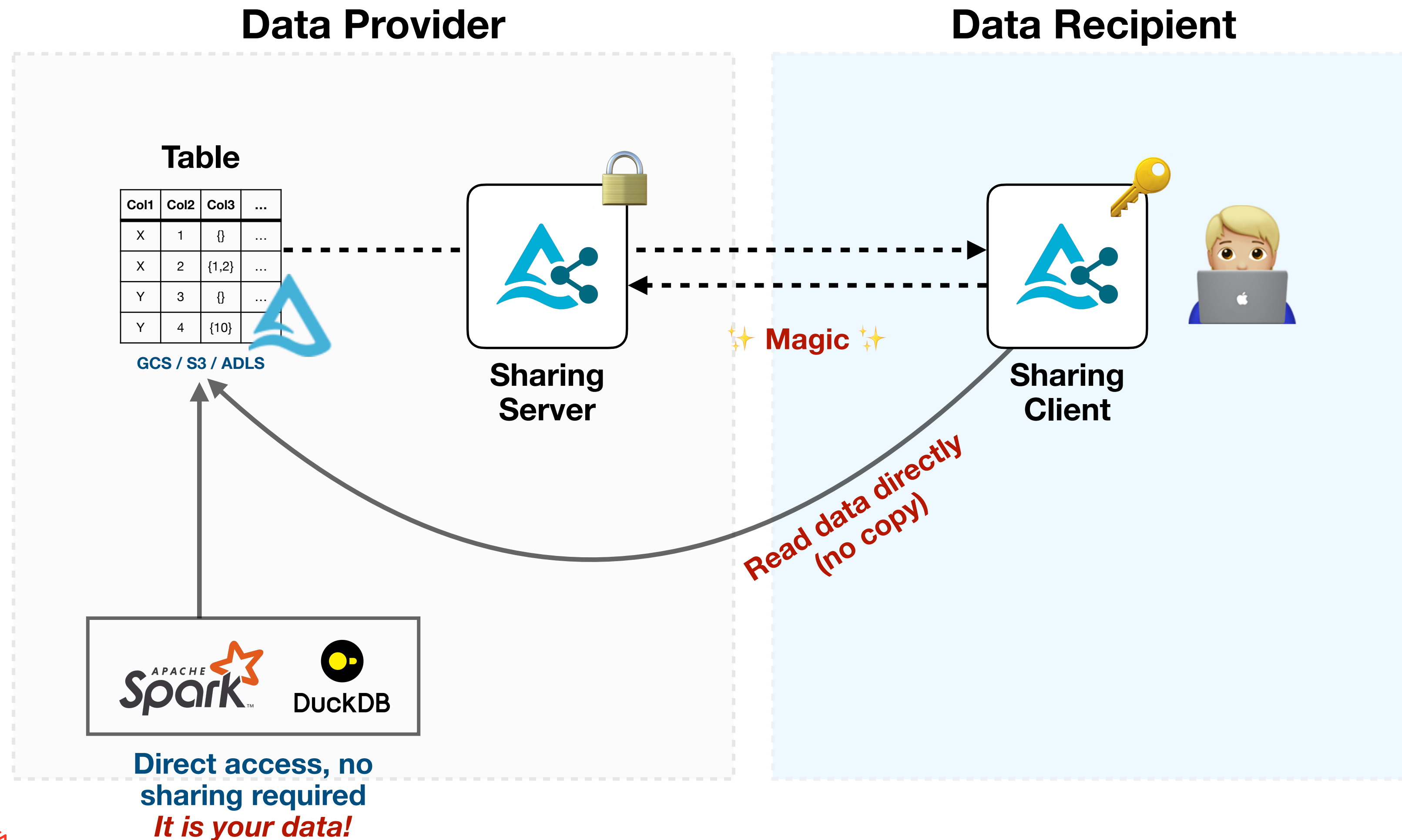


- Provider governs access to tables
- Recipient decides which client they want to use
- Does not change access patterns for provider
- Recipients are given controlled, temporary access to data which means no duplication of data



High level overview

Zero-copy data access



- Provider governs access to tables
- Recipient decides which client they want to use
- Does not change access patterns for provider
- Recipients are given controlled, temporary access to data which means no duplication of data
- Shared assets are READ ONLY



Open Source Clients



Business Intelligence / Analytics



Hyperscalers



Governance



SaaS / Multi-Cloud Infrastructure



3rd Party Data Vendors / Clean Room



Open Source Clients



Business Intelligence / Analytics



Governance



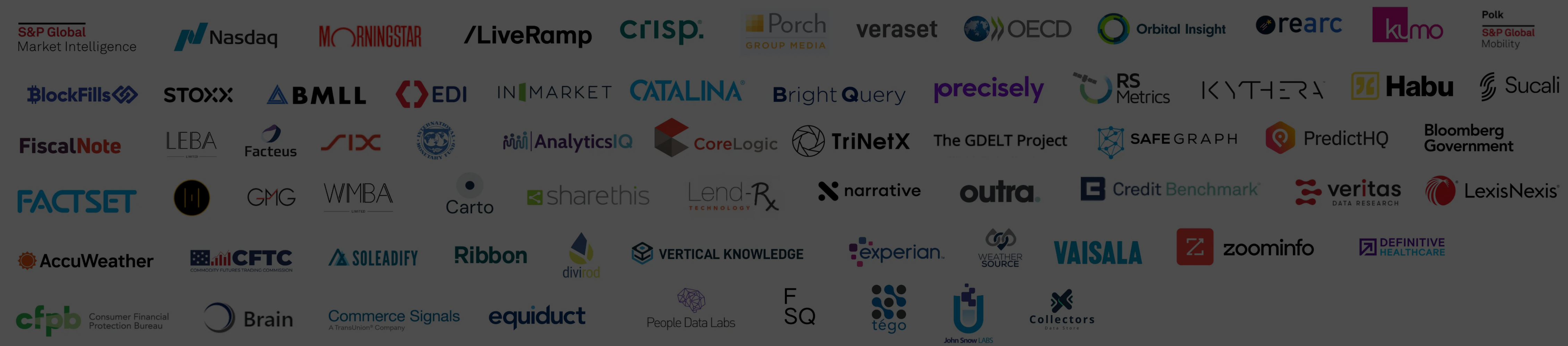
Hyperscalers



SaaS / Multi-Cloud Infrastructure



3rd Party Data Vendors / Clean Room



Example: Pandas



Example: Pandas

```
import delta_sharing as ds

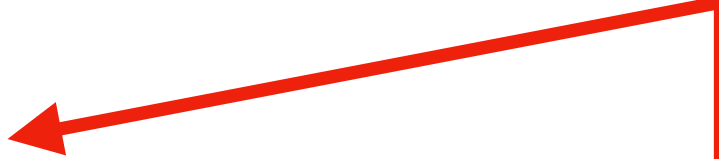
profile_file = "open-datasets.share"
client = ds.SharingClient(profile=profile_file)
```



Example: Pandas

```
import delta_sharing as ds

profile_file = "open-datasets.share"
client = ds.SharingClient(profile=profile_file)
```



```
{
  "shareCredentialsVersion": 1,
  "endpoint": "https://sharing.delta.io/delta-sharing/",
  "bearerToken": "faaie590d541265bcab1f2de9813274bf233"
}
```



Example: Pandas

```
import delta_sharing as ds
```

```
profile_file = "open-datasets.share"
```

```
client = ds.SharingClient(profile=profile_file)
```

```
client.list_all_tables()
```

```
[Table(name='COVID_19_NYT', share='delta_sharing', schema='default'),  
Table(name='boston-housing', share='delta_sharing', schema='default'),  
Table(name='flight-asa_2008', share='delta_sharing', schema='default'),  
Table(name='lending_club', share='delta_sharing', schema='default'),  
Table(name='nyctaxi_2019', share='delta_sharing', schema='default'),  
Table(name='nyctaxi_2019_part', share='delta_sharing', schema='default'),  
Table(name='owid-covid-data', share='delta_sharing', schema='default')]
```

```
{  
  "shareCredentialsVersion": 1,  
  "endpoint": "https://sharing.delta.io/delta-sharing/",  
  "bearerToken": "faaie590d541265bcab1f2de9813274bf233"  
}
```



Example: Pandas

```
import delta_sharing as ds
```

```
profile_file = "open-datasets.share"
```

```
client = ds.SharingClient(profile=profile_file)
```

```
client.list_all_tables()
```

```
[Table(name='COVID_19_NYT', share='delta_sharing', schema='default'),  
Table(name='boston-housing', share='delta_sharing', schema='default'),  
Table(name='flight-asa_2008', share='delta_sharing', schema='default'),  
Table(name='lending_club', share='delta_sharing', schema='default'),  
Table(name='nyctaxi_2019', share='delta_sharing', schema='default'),  
Table(name='nyctaxi_2019_part', share='delta_sharing', schema='default'),  
Table(name='owid-covid-data', share='delta_sharing', schema='default')]
```

```
{  
  "shareCredentialsVersion": 1,  
  "endpoint": "https://sharing.delta.io/delta-sharing/",  
  "bearerToken": "faaie590d541265bcab1f2de9813274bf233"  
}
```



Example: Pandas

```
import delta_sharing as ds
```

```
profile_file = "open-datasets.share"
```

```
client = ds.SharingClient(profile=profile_file)
```

```
client.list_all_tables()
```

```
[Table(name='COVID_19_NYT', share='delta_sharing', schema='default'),  
Table(name='boston-housing', share='delta_sharing', schema='default'),  
Table(name='flight-asa_2008', share='delta_sharing', schema='default'),  
Table(name='lending_club', share='delta_sharing', schema='default'),  
Table(name='nyctaxi_2019', share='delta_sharing', schema='default'),  
Table(name='nyctaxi_2019_part', share='delta_sharing', schema='default'),  
Table(name='owid-covid-data', share='delta_sharing', schema='default')]
```

```
table_url = profile_file + "#delta_sharing.default.COVID_19_NYT"
```

```
sample_df = ds.load_as_pandas(table_url, limit=10)
```

```
sample_df
```

```
{  
  "shareCredentialsVersion": 1,  
  "endpoint": "https://sharing.delta.io/delta-sharing/",  
  "bearerToken": "faaie590d541265bcab1f2de9813274bf233"  
}
```



Example: Pandas

```
import delta_sharing as ds

profile_file = "open-datasets.share"
client = ds.SharingClient(profile=profile_file)

client.list_all_tables()
[Table(name='COVID_19_NYT', share='delta_sharing', schema='default'),
 Table(name='boston-housing', share='delta_sharing', schema='default'),
 Table(name='flight-asa_2008', share='delta_sharing', schema='default'),
 Table(name='lending_club', share='delta_sharing', schema='default'),
 Table(name='nyctaxi_2019', share='delta_sharing', schema='default'),
 Table(name='nyctaxi_2019_part', share='delta_sharing', schema='default'),
 Table(name='owid-covid-data', share='delta_sharing', schema='default')]

table_url = profile_file + "#delta_sharing.default.COVID_19_NYT"
sample_df = ds.load_as_pandas(table_url, limit=10)
sample_df
```

```
{
  "shareCredentialsVersion": 1,
  "endpoint": "https://sharing.delta.io/delta-sharing/",
  "bearerToken": "faaie590d541265bcab1f2de9813274bf233"
}
```

	date	county	state	fips	cases	deaths
0	2021-01-10	Washakie	Wyoming	56043	804	21
1	2021-01-10	Weston	Wyoming	56045	485	4
2	2021-01-11	Autauga	Alabama	1001	4902	55
3	2021-01-11	Baldwin	Alabama	1003	15417	173
4	2021-01-11	Barbour	Alabama	1005	1663	35



How Delta Works

Just enough to understand Delta Sharing



This example deliberately omits features like deletion vectors to keep things simple

How Delta Works

Just enough to understand Delta Sharing

Col1	Col2	Col3	...
X	1	{}	...
X	2	{1,2}	...
Y	3	{}	...
Y	4	{10}	...



This example deliberately omits features like deletion vectors to keep things simple

How Delta Works

Just enough to understand Delta Sharing

Col1	Col2	Col3	...
X	1	{}	...
X	2	{1,2}	...
Y	3	{}	...
Y	4	{10}	...



`/<table-name>/_delta_log/`

↳ `000000000001.json`

`/<table-name>/_change_data/`

↳ `cdc-0000001.parquet`

`0000001.parquet`



How Delta Works

Just enough to understand Delta Sharing

Col1	Col2	Col3	...
X	1	{}	...
X	2	{1,2}	...
Y	3	{}	...
Y	4	{10}	...

+

Col1	Col2	Col3	...
X	5	{99}	...
X	6	{}	...
Y	7	{}	...
Y	8	{12, 35}	...

/<table-name>/_delta_log/

↳ 000000000001.json

/<table-name>/_change_data/

↳ cdc-0000001.parquet

0000001.parquet



How Delta Works

Just enough to understand Delta Sharing

Col1	Col2	Col3	...
X	1	{}	...
X	2	{1,2}	...
Y	3	{}	...
Y	4	{10}	...

+

Col1	Col2	Col3	...
X	5	{99}	...
X	6	{}	...
Y	7	{}	...
Y	8	{12, 35}	...

/<table-name>/_delta_log/

↳ 000000000001.json

↳ 000000000002.json

↳ 000000000002.checkpoint.parquet

/<table-name>/_change_data/

↳ cdc-0000001.parquet

↳ cdc-0000002.parquet

0000001.parquet

0000002.parquet

0000003.parquet



How Delta Works

Just enough to understand Delta Sharing

Col1	Col2	Col3	...
X	1	{}	...
X	2	{1,2}	...
Y	3	{}	...
Y	4	{10}	...

+

Col1	Col2	Col3	...
X	5	{99}	...
X	6	{}	...
Y	7	{}	...
Y	8	{12, 35}	...

```
/<table-name>/_delta_log/  
  ↳ 000000000001.json  
  ↳ 000000000002.json  
  ↳ 000000000002.checkpoint.parquet  
  
/<table-name>/_change_data/  
  ↳ cdc-0000001.parquet  
  ↳ cdc-0000002.parquet  
  
0000001.parquet  
0000002.parquet  
0000003.parquet
```

Delta Log

“What happened”
E.g. Add 1 file, Remove 5 files



This example deliberately omits features like deletion vectors to keep things simple

How Delta Works

Just enough to understand Delta Sharing

Col1	Col2	Col3	...
X	1	{}	...
X	2	{1,2}	...
Y	3	{}	...
Y	4	{10}	...

+

Col1	Col2	Col3	...
X	5	{99}	...
X	6	{}	...
Y	7	{}	...
Y	8	{12, 35}	...

/<table-name>/_delta_log/

↳ 000000000001.json

↳ 000000000002.json

↳ 000000000002.checkpoint.parquet

/<table-name>/_change_data/

↳ cdc-0000001.parquet

↳ cdc-0000002.parquet

0000001.parquet

0000002.parquet

0000003.parquet

Delta Log

“What happened”
E.g. Add 1 file, Remove 5 files

Change Data Feed

CDC-style data



How Delta Works

Just enough to understand Delta Sharing

Col1	Col2	Col3	...
X	1	{}	...
X	2	{1,2}	...
Y	3	{}	...
Y	4	{10}	...

+

Col1	Col2	Col3	...
X	5	{99}	...
X	6	{}	...
Y	7	{}	...
Y	8	{12, 35}	...

/<table-name>/_delta_log/

↳ 000000000001.json

↳ 000000000002.json

↳ 000000000002.checkpoint.parquet

/<table-name>/_change_data/

↳ cdc-0000001.parquet

↳ cdc-0000002.parquet

0000001.parquet

0000002.parquet

0000003.parquet

Delta Log

“What happened”
E.g. Add 1 file, Remove 5 files

Change Data Feed

CDC-style data

Data

Self explanatory
Not all files are part of current table



This example deliberately omits features like deletion vectors to keep things simple

Also works for change data feed

Pull incremental changes to data as required

```
# Read changes for a specific version  
ds.load_table_changes_as_pandas(table_url, starting_version=3, ending_version=3)
```



Also works for change data feed

Pull incremental changes to data as required

```
# Read changes for a specific version
```

```
ds.load_table_changes_as_pandas(table_url, starting_version=3, ending_version=3)
```



Also works for change data feed

Pull incremental changes to data as required

```
# Read changes for a specific version
ds.load_table_changes_as_pandas(table_url, starting_version=3, ending_version=3)
```

	inventory_id	product_sku	product_name	product_category	warehouse_id	warehouse_region	supplier_name	quantity_on_hand	unit_cost	unit_price	inventory_status	last_received_date	_change_type	_commit_version	_commit_timestamp
0	1128326	SKU-128326	Beverages Max T7	Food & Beverage	WH-0127	South Asia	Harbor Group	9168	34.70	57.26	Backordered	2025-10-04	update_postimage	3	1775785178000
1	1128326	SKU-128326	Beverages Max T7	Food & Beverage	WH-0127	South Asia	Harbor Group	9168	34.70	57.26	On Order	2025-10-04	update_preimage	3	1775785178000
2	1128439	SKU-128439	Imaging Select Q10	Medical Devices	WH-0040	Oceania	Quantum Trading	2590	294.06	352.87	Backordered	2025-05-26	update_postimage	3	1775785178000
3	1128439	SKU-128439	Imaging Select Q10	Medical Devices	WH-0040	Oceania	Quantum Trading	2590	294.06	352.87	On Order	2025-05-26	update_preimage	3	1775785178000
4	1128554	SKU-128554	Snacks Heavy-Duty Series-5	Food & Beverage	WH-0155	Asia Pacific	Alpha Limited	2877	431.60	556.76	Backordered	2025-10-18	update_postimage	3	1775785178000
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
555827	17181866	SKU-181866	Snacks Max T7	Food & Beverage	WH-0067	Asia Pacific	Harbor Group	8478	25.02	47.04	On Order	2025-01-11	update_preimage	3	1775785178000
555828	17181907	SKU-181907	Surgical Lite A800	Medical Devices	WH-0108	Latin America	Titan Partners	9750	205.37	303.95	Backordered	2025-12-05	update_postimage	3	1775785178000
555829	17181907	SKU-181907	Surgical Lite A800	Medical Devices	WH-0108	Latin America	Titan Partners	9750	205.37	303.95	On Order	2025-12-05	update_preimage	3	1775785178000
555830	17181890	SKU-181890	Dairy Eco X100	Food & Beverage	WH-0091	Asia Pacific	Stellar Logistics	5879	22.92	27.05	Backordered	2024-02-15	update_postimage	3	1775785178000
555831	17181890	SKU-181890	Dairy Eco X100	Food & Beverage	WH-0091	Asia Pacific	Stellar Logistics	5879	22.92	27.05	On Order	2024-02-15	update_preimage	3	1775785178000



Also works for change data feed

Pull incremental changes to data as required

```
# Read changes for a specific version
ds.load_table_changes_as_pandas(table_url, starting_version=3, ending_version=3)
```

	unit_cost	unit_price	inventory_status	last_received_date	_change_type	_commit_version	_commit_timestamp
	34.70	57.26	Backordered	2025-10-04	update_postimage	3	1775785178000
	34.70	57.26	On Order	2025-10-04	update_preimage	3	1775785178000
	294.06	352.87	Backordered	2025-05-26	update_postimage	3	1775785178000
	294.06	352.87	On Order	2025-05-26	update_preimage	3	1775785178000
	431.60	556.76	Backordered	2025-10-18	update_postimage	3	1775785178000
	...	...	...	...	...	...	...
	25.02	47.04	On Order	2025-01-11	update_preimage	3	1775785178000

- Rows for before/after update etc
- Three extra columns



Also works for change data feed

Pull incremental changes to data as required

```
# Read changes for a specific version
ds.load_table_changes_as_pandas(table_url, starting_version=3, ending_version=3)
```

and	unit_cost	unit_price	inventory_status	last_received_date	_change_type	_commit_version	_commit_timestamp
	34.70	57.26	Backordered	2025-10-04	update_postimage	3	1775785178000
	34.70	57.26	On Order	2025-10-04	update_preimage	3	1775785178000
	294.06	352.87	Backordered	2025-05-26	update_postimage	3	1775785178000
	294.06	352.87	On Order	2025-05-26	update_preimage	3	1775785178000
	431.60	556.76	Backordered	2025-10-18	update_postimage	3	1775785178000
	...	...	...	...	...	...	...
	25.02	47.04	On Order	2025-01-11	update_preimage	3	1775785178000

- Rows for before/after update etc
- Three extra columns

```
# other ways to query changes:
ds.load_table_changes_as_pandas(table_url, starting_version=1)
ds.load_table_changes_as_pandas(table_url, starting_version=3, ending_version=4)
ds.load_table_changes_as_pandas(
    table_url,
    starting_timestamp="2026-04-10T00:00:00Z",
    ending_timestamp="2026-04-11T00:00:00Z",
)
```



Also works for change data feed

Pull incremental changes to data as required

```
# Read changes for a specific version
ds.load_table_changes_as_pandas(table_url, starting_version=3, ending_version=3)
```

and	unit_cost	unit_price	inventory_status	last_received_date	_change_type	_commit_version	_commit_timestamp
	34.70	57.26	Backordered	2025-10-04	update_postimage	3	1775785178000
	34.70	57.26	On Order	2025-10-04	update_preimage	3	1775785178000
	294.06	352.87	Backordered	2025-05-26	update_postimage	3	1775785178000
	294.06	352.87	On Order	2025-05-26	update_preimage	3	1775785178000
	431.60	556.76	Backordered	2025-10-18	update_postimage	3	1775785178000
	...	...	...	...	...	...	...
	25.02	47.04	On Order	2025-01-11	update_preimage	3	1775785178000

- Rows for before/after update etc
- Three extra columns

```
# other ways to query changes:
ds.load_table_changes_as_pandas(table_url, starting_version=1)
ds.load_table_changes_as_pandas(table_url, starting_version=3, ending_version=4)
ds.load_table_changes_as_pandas(
    table_url,
    starting_timestamp="2026-04-10T00:00:00Z",
    ending_timestamp="2026-04-11T00:00:00Z",
)
```



Also works for change data feed

Pull incremental changes to data as required

```
# Read changes for a specific version
ds.load_table_changes_as_pandas(table_url, starting_version=3, ending_version=3)
```

and	unit_cost	unit_price	inventory_status	last_received_date	_change_type	_commit_version	_commit_timestamp
	34.70	57.26	Backordered	2025-10-04	update_postimage	3	1775785178000
	34.70	57.26	On Order	2025-10-04	update_preimage	3	1775785178000
	294.06	352.87	Backordered	2025-05-26	update_postimage	3	1775785178000
	294.06	352.87	On Order	2025-05-26	update_preimage	3	1775785178000
	431.60	556.76	Backordered	2025-10-18	update_postimage	3	1775785178000
	...	...	...	...	...	...	...
	25.02	47.04	On Order	2025-01-11	update_preimage	3	1775785178000

- Rows for before/after update etc
- Three extra columns

```
# other ways to query changes:
ds.load_table_changes_as_pandas(table_url, starting_version=1)
ds.load_table_changes_as_pandas(table_url, starting_version=3, ending_version=4)
ds.load_table_changes_as_pandas(
    table_url,
    starting_timestamp="2026-04-10T00:00:00Z",
    ending_timestamp="2026-04-11T00:00:00Z",
)
```



Also works for change data feed

Pull incremental changes to data as required

```
# Read changes for a specific version
```

```
ds.load_table_changes_as_pandas(table_url, starting_version=3, ending_version=3)
```

and	unit_cost	unit_price	inventory_status	last_received_date	_change_type	_commit_version	_commit_timestamp
	34.70	57.26	Backordered	2025-10-04	update_postimage	3	1775785178000
	34.70	57.26	On Order	2025-10-04	update_preimage	3	1775785178000
	294.06	352.87	Backordered	2025-05-26	update_postimage	3	1775785178000
	294.06	352.87	On Order	2025-05-26	update_preimage	3	1775785178000
	431.60	556.76	Backordered	2025-10-18	update_postimage	3	1775785178000
	...	...	...	...	...	...	...
	25.02	47.04	On Order	2025-01-11	update_preimage	3	1775785178000

- Rows for before/after update etc
- Three extra columns

```
# other ways to query changes:
```

```
ds.load_table_changes_as_pandas(table_url, starting_version=1)
```

```
ds.load_table_changes_as_pandas(table_url, starting_version=3, ending_version=4)
```

```
ds.load_table_changes_as_pandas(  
    table_url,  
    starting_timestamp="2026-04-10T00:00:00Z",  
    ending_timestamp="2026-04-11T00:00:00Z",  
)
```

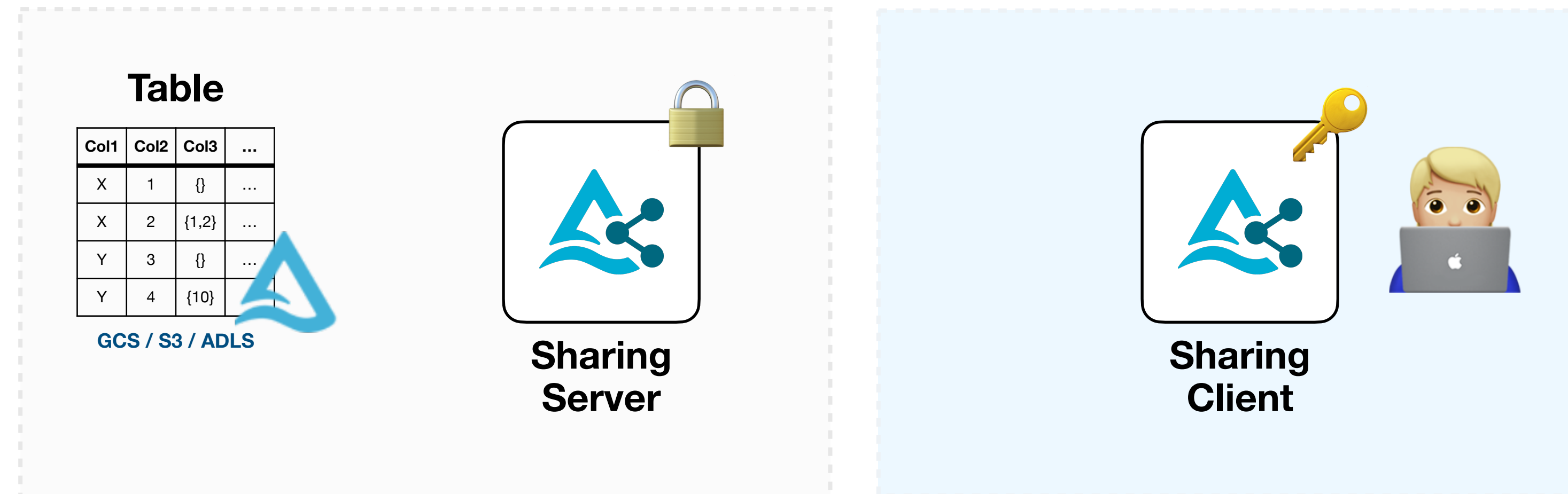
No need to retrieve whole table again, improved efficiency.

Smallest increment to pull is a single version.



How do the client & server interact?

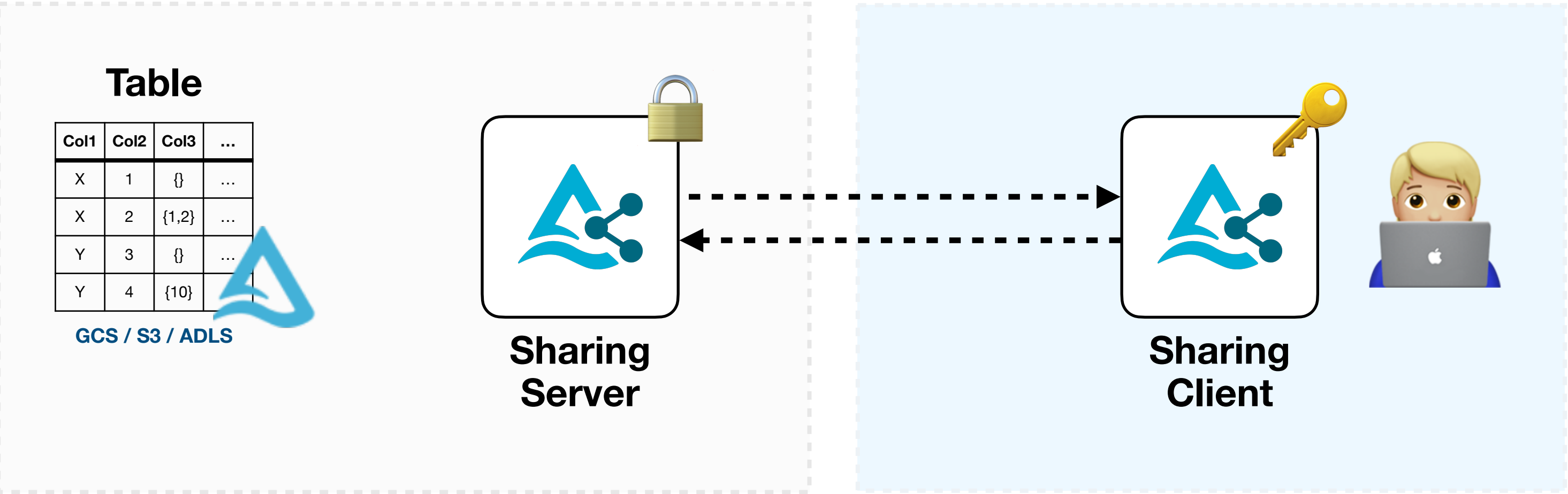
What *actually* happens



How do the client & server interact?

What *actually* happens

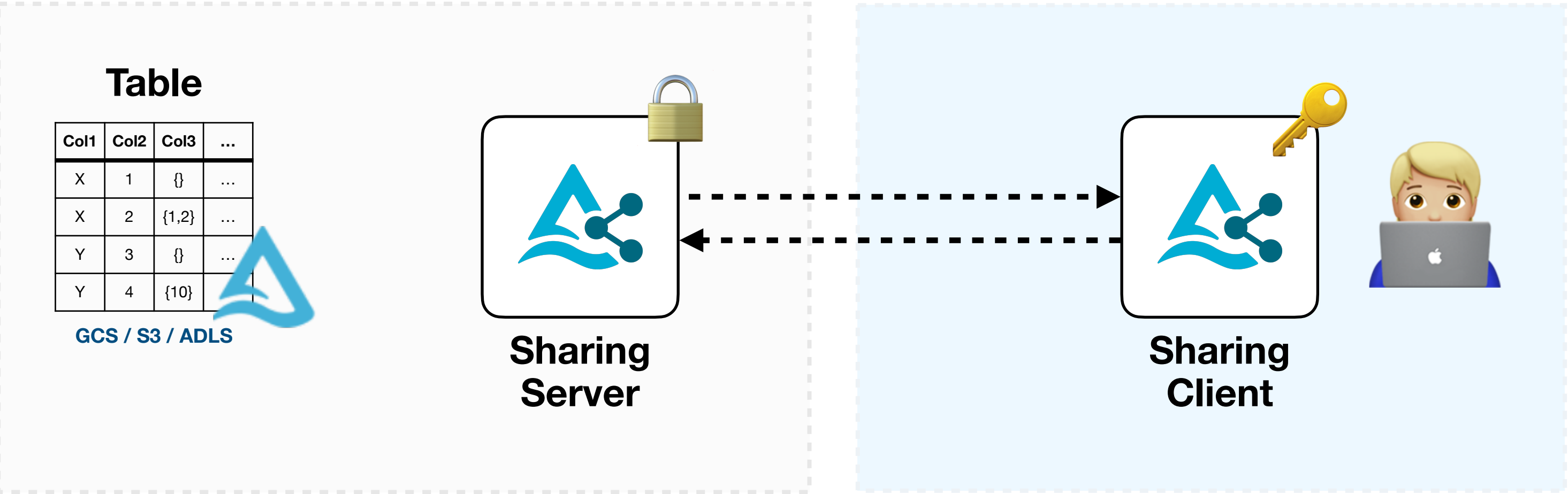
What is available?



How do the client & server interact?

What *actually* happens

What is available?



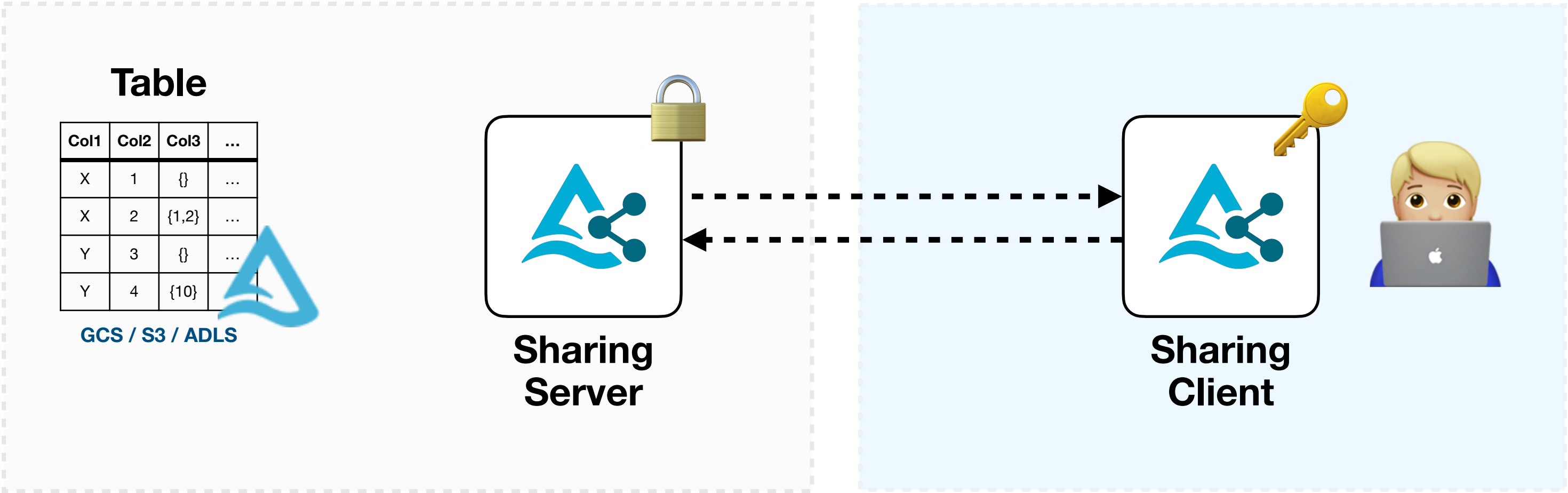
```
{
  "items": [
    {
      "name": "<table>",
      "schema": "<schema>",
      "share": "<share-name>",
      "share_id": "4e163799-f717-4127-8ga4-6f548b4835a6",
      "id": "61c2d2f3-6d09-39ad-8bc1-66d8283bb7d5"
    }
  ]
}
```



How do the client & server interact?

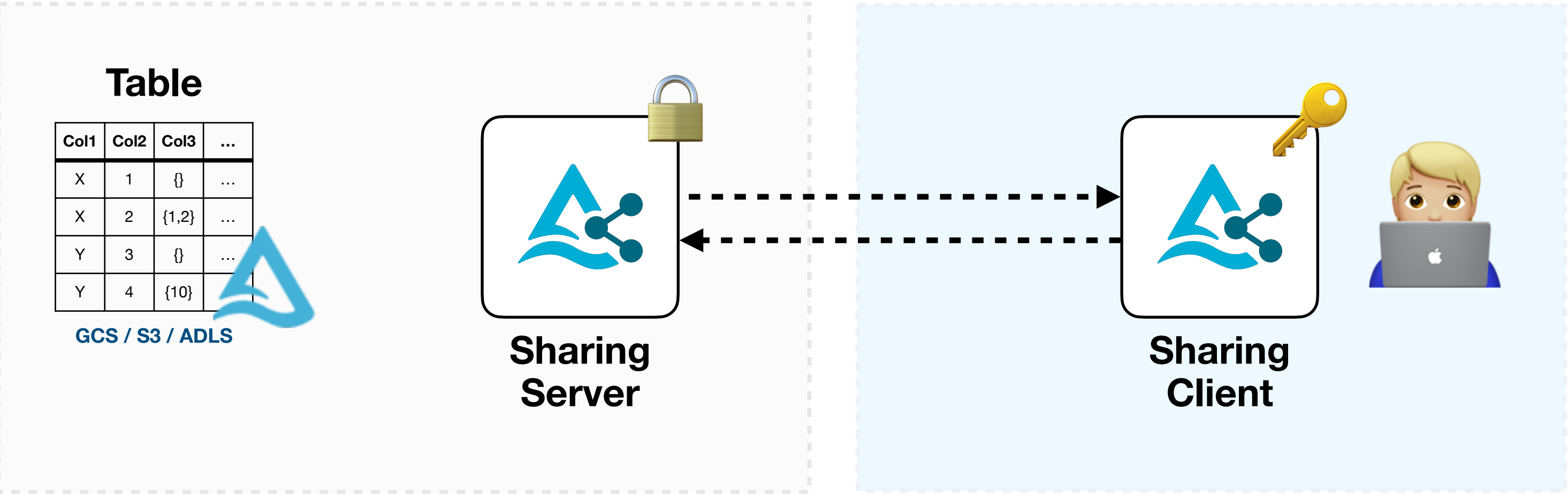
What *actually* happens

What do you know about this table?



How do the client & server interact?

What *actually* happens



What do you know about this table?

```
{
  "protocol": {
    "minReaderVersion": 1
  }
}

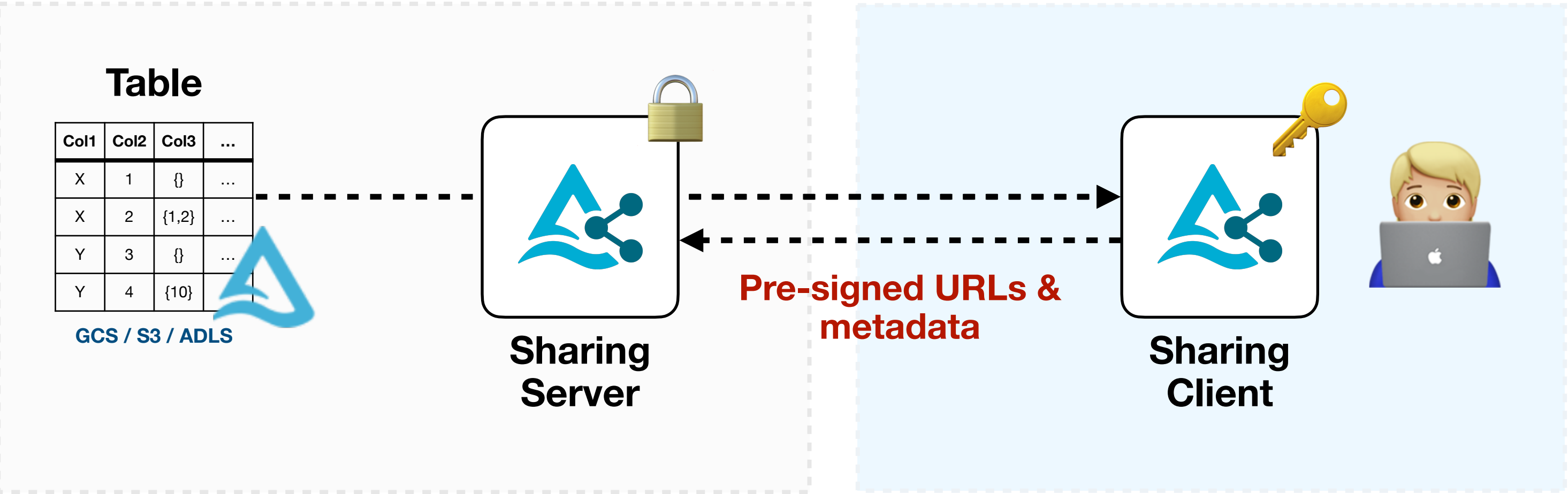
"metadata": {
  "id": "61a2d2f3-6d09-49ad-8bc1-66d8283aa5d5",
  "format": { "provider": "parquet" },
  "schemaString": {
    "type": "struct",
    "fields": [
      { "name": "inventory_id", "type": "long", ... },
      { "name": "product_sku", "type": "string", ... },
      { "name": "product_name", "type": "string", ... },
      { "name": "product_category", "type": "string", ... },
      ...
    ]
  },
  "partitionColumns": [],
  "configuration": { "enableChangeDataFeed": "true" },
  "size": 235633227,
  "numFiles": 4
}
```



How do the client & server interact?

What *actually* happens

What do you know about this table?



```
{
  "protocol": {
    "minReaderVersion": 1
  }
}

{
  "metaData": {
    "id": "61a2d2f3-6d09-49ad-8bc1-66d8283aa5d5",
    "format": { "provider": "parquet" },
    "schemaString": {
      "type": "struct",
      "fields": [
        { "name": "inventory_id", "type": "long", ... },
        { "name": "product_sku", "type": "string", ... },
        { "name": "product_name", "type": "string", ... },
        { "name": "product_category", "type": "string", ... },
        ...
      ]
    },
    "partitionColumns": [],
    "configuration": { "enableChangeDataFeed": "true" },
    "size": 235633227,
    "numFiles": 4
  }
}
```

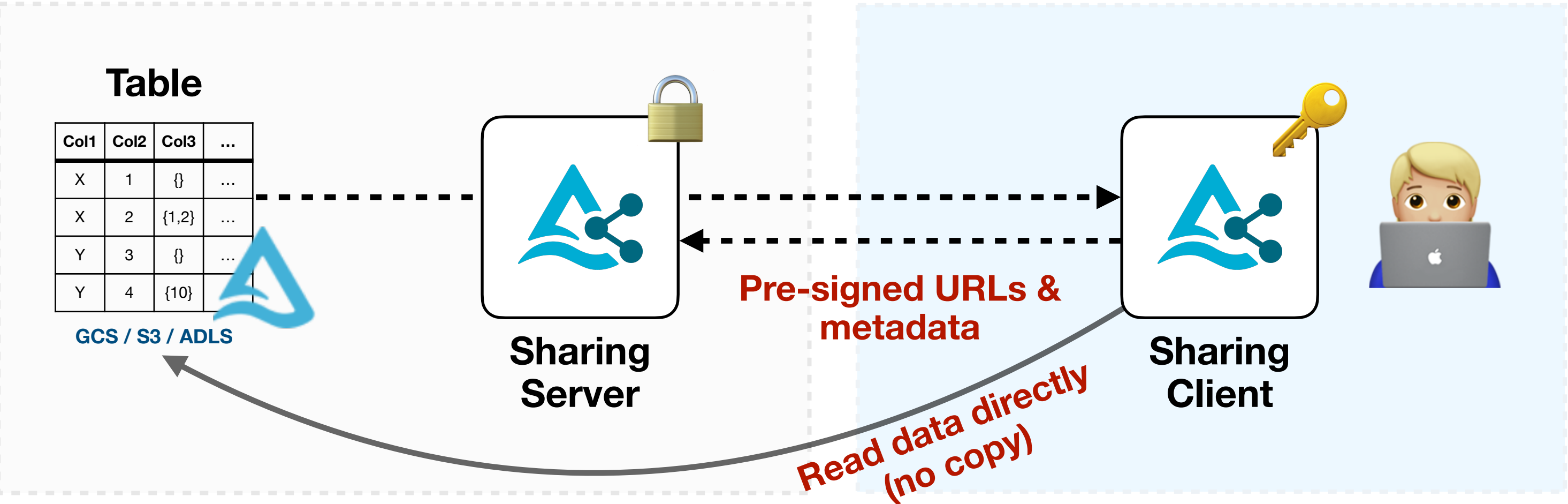
One of many!



How do the client & server interact?

What *actually* happens

What do you know about this table?



```
{
  "protocol": {
    "minReaderVersion": 1
  }
  "metaData": {
    "id": "61a2d2f3-6d09-49ad-8bc1-66d8283aa5d5",
    "format": { "provider": "parquet" },
    "schemaString": {
      "type": "struct",
      "fields": [
        { "name": "inventory_id", "type": "long", ... },
        { "name": "product_sku", "type": "string", ... },
        { "name": "product_name", "type": "string", ... },
        { "name": "product_category", "type": "string", ... },
        ...
      ]
    },
    "partitionColumns": [],
    "configuration": { "enableChangeDataFeed": "true" },
    "size": 235633227,
    "numFiles": 4
  }
}
```

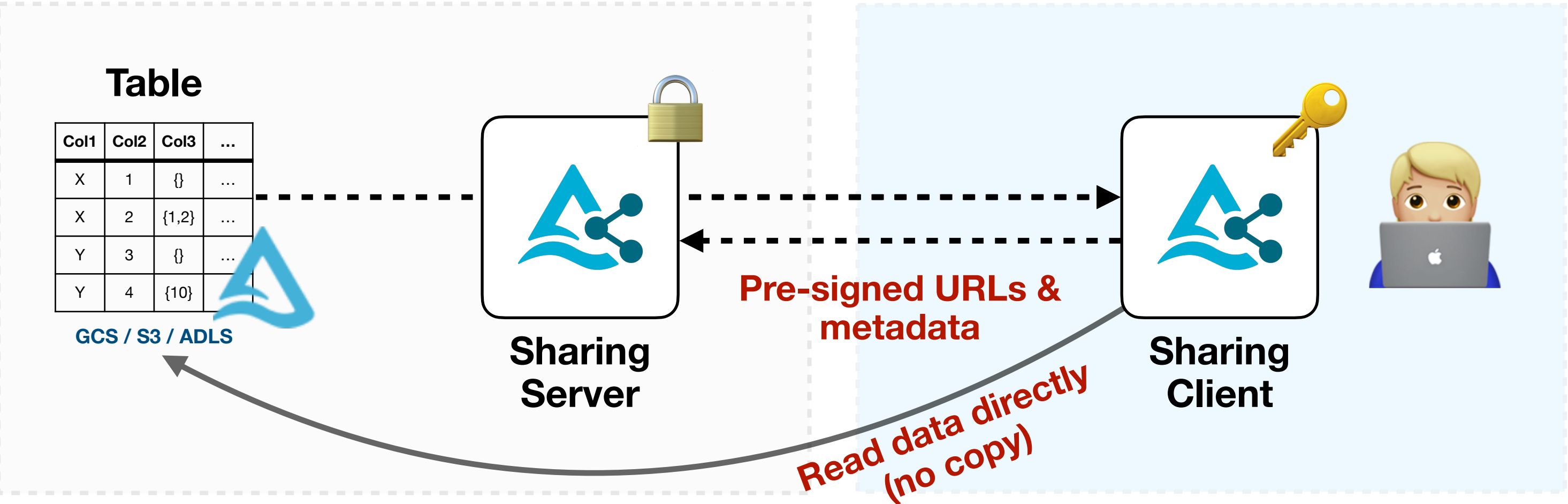
One of many!



How do the client & server interact?

What *actually* happens

What do you know about this table?



```
{
  "protocol": {
    "minReaderVersion": 1
  }
  "metaData": {
    "id": "61a2d2f3-6d09-49ad-8bc1-66d8283aa5d5",
    "format": { "provider": "parquet" },
    "schemaString": {
      "type": "struct",
      "fields": [
        { "name": "inventory_id", "type": "long", ... },
        { "name": "product_sku", "type": "string", ... },
        { "name": "product_name", "type": "string", ... },
        { "name": "product_category", "type": "string", ... },
        ...
      ]
    },
    "partitionColumns": [],
    "configuration": { "enableChangeDataFeed": "true" },
    "size": 235633227,
    "numFiles": 4
  }
}
```

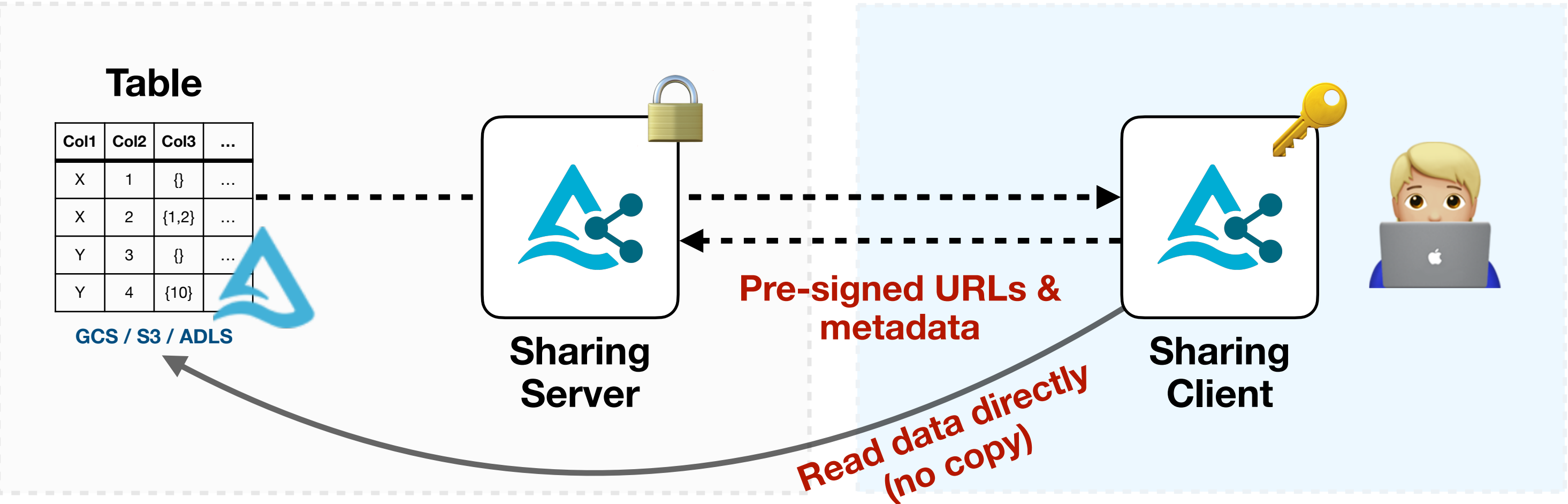
One of many!

```
{
  "file": {
    "url": "https://<cloud-storage-bucket>/.../<file-name>.zstd.parquet?<presigned-query-params>",
    "expirationTimestamp": 1775802138000,
    "id": "626b099bb94c90d5e4960bed21ea0597",
    "partitionValues": {},
    "size": 56778672,
    "stats": {
      "numRecords": 4993771,
      "minValues": { "inventory_id": 10000001, "...": "... " },
      "maxValues": { "inventory_id": 19999990, "...": "... " },
      "nullCount": { "inventory_id": 0, "...": 0 }
    }
  }
}
```



How do the client & server interact?

What *actually* happens

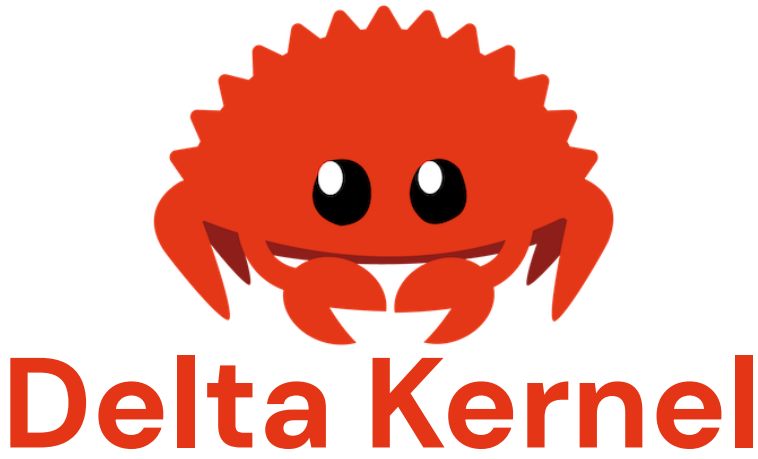


What do you know about this table?

```
{
  "protocol": {
    "minReaderVersion": 1
  }
  "metaData": {
    "id": "61a2d2f3-6d09-49ad-8bc1-66d8283aa5d5",
    "format": { "provider": "parquet" },
    "schemaString": {
      "type": "struct",
      "fields": [
        { "name": "inventory_id", "type": "long", ... },
        { "name": "product_sku", "type": "string", ... },
        { "name": "product_name", "type": "string", ... },
        { "name": "product_category", "type": "string", ... },
        ...
      ]
    },
    "partitionColumns": [],
    "configuration": { "enableChangeDataFeed": "true" },
    "size": 235633227,
    "numFiles": 4
  }
}
```

One of many!

```
{
  "file": {
    "url": "https://<cloud-storage-bucket>/.../<file-name>.zstd.parquet?<presigned-query-params>",
    "expirationTimestamp": 1775802138000,
    "id": "626b099bb94c90d5e4960bed21ea0597",
    "partitionValues": {},
    "size": 56778672,
    "stats": {
      "numRecords": 4993771,
      "minValues": { "inventory_id": 10000001, "...": "... " },
      "maxValues": { "inventory_id": 19999990, "...": "... " },
      "nullCount": { "inventory_id": 0, "...": 0 }
    }
  }
}
```



Delta Kernel



Delta Kernel

Challenge:

Client needs to reconstruct table using the metadata from the server. This includes support for complex features like deletion vectors.



Delta Kernel

Challenge:

Client needs to reconstruct table using the metadata from the server. This includes support for complex features like deletion vectors.

Solution:

Save the metadata as a quasi delta log and retrieve via Delta Kernel as Arrow batches.



Delta Kernel

Challenge:

Client needs to reconstruct table using the metadata from the server. This includes support for complex features like deletion vectors.

Solution:

Save the metadata as a quasi delta log and retrieve via Delta Kernel as Arrow batches.

Delta Kernel

The Delta Kernel project is a set of libraries ([Java](#) and [Rust](#)) for building Delta connectors that can read from and write into Delta tables without the need to understand the [Delta protocol details](#).

You can use this library to do the following:

- Read data from small Delta tables in a single thread in a single process.
- Read data from large Delta tables using multiple threads in a single process.
- Build a complex connector for a distributed processing engine and read very large Delta tables.
- Insert data into a Delta table either from a single process or a complex distributed engine.



Delta Kernel

The Delta Kernel project is a set of libraries ([Java](#) and [Rust](#)) for building Delta connectors that can read from and write into Delta tables without the need to understand the [Delta protocol details](#).

You can use this library to do the following:

- Read data from small Delta tables in a single thread in a single process.
- Read data from large Delta tables using multiple threads in a single process.
- Build a complex connector for a distributed processing engine and read very large Delta tables.
- Insert data into a Delta table either from a single process or a complex distributed engine.



Delta Kernel

Challenge:

Client needs to reconstruct table using the metadata from the server. This includes support for complex features like deletion vectors.

Solution:

Save the metadata as a quasi delta log and retrieve via Delta Kernel as Arrow batches.

Delta Kernel

The Delta Kernel project is a set of libraries ([Java](#) and [Rust](#)) for building Delta connectors that can read from and write into Delta tables without the need to understand the [Delta protocol details](#).

You can use this library to do the following:

- Read data from small Delta tables in a single thread in a single process.
- Read data from large Delta tables using multiple threads in a single process.
- Build a complex connector for a distributed processing engine and read very large Delta tables.
- Insert data into a Delta table either from a single process or a complex distributed engine.



Delta Kernel

Challenge:

Client needs to reconstruct table using the metadata from the server. This includes support for complex features like deletion vectors.

Solution:

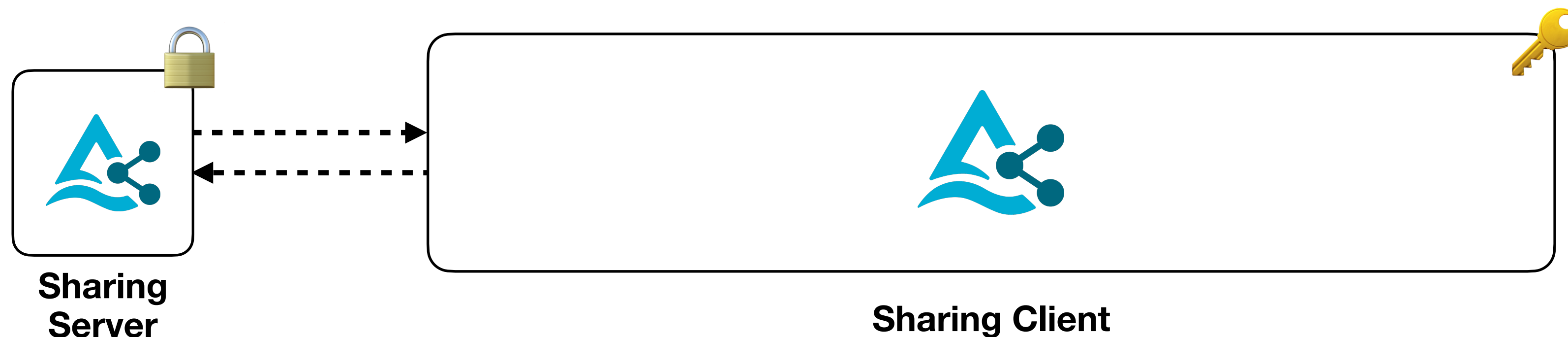
Save the metadata as a quasi delta log and retrieve via Delta Kernel as Arrow batches.

Delta Kernel

The Delta Kernel project is a set of libraries ([Java](#) and [Rust](#)) for building Delta connectors that can read from and write into Delta tables without the need to understand the [Delta protocol details](#).

You can use this library to do the following:

- Read data from small Delta tables in a single thread in a single process.
- Read data from large Delta tables using multiple threads in a single process.
- Build a complex connector for a distributed processing engine and read very large Delta tables.
- Insert data into a Delta table either from a single process or a complex distributed engine.



Delta Kernel

Challenge:

Client needs to reconstruct table using the metadata from the server. This includes support for complex features like deletion vectors.

Solution:

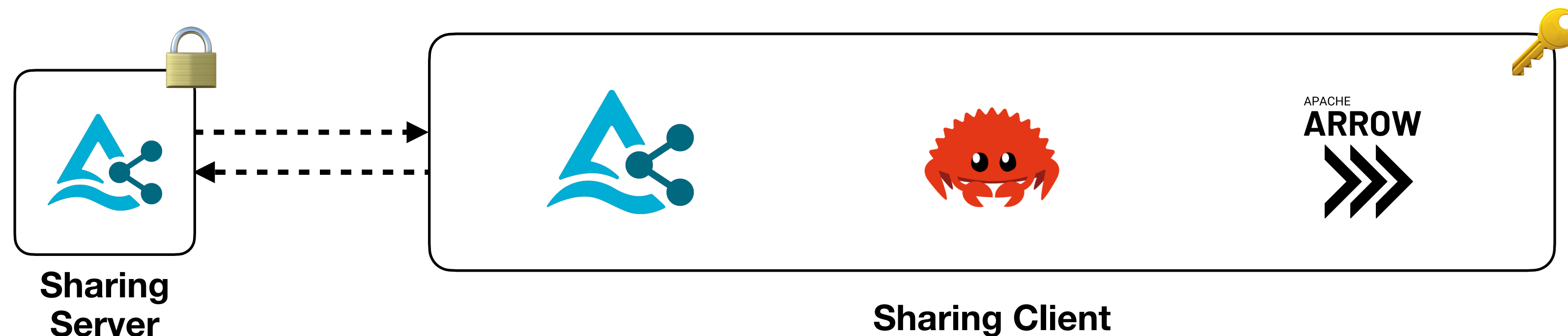
Save the metadata as a quasi delta log and retrieve via Delta Kernel as Arrow batches.

Delta Kernel

The Delta Kernel project is a set of libraries ([Java](#) and [Rust](#)) for building Delta connectors that can read from and write into Delta tables without the need to understand the [Delta protocol details](#).

You can use this library to do the following:

- Read data from small Delta tables in a single thread in a single process.
- Read data from large Delta tables using multiple threads in a single process.
- Build a complex connector for a distributed processing engine and read very large Delta tables.
- Insert data into a Delta table either from a single process or a complex distributed engine.



Delta Kernel

Challenge:

Client needs to reconstruct table using the metadata from the server. This includes support for complex features like deletion vectors.

Solution:

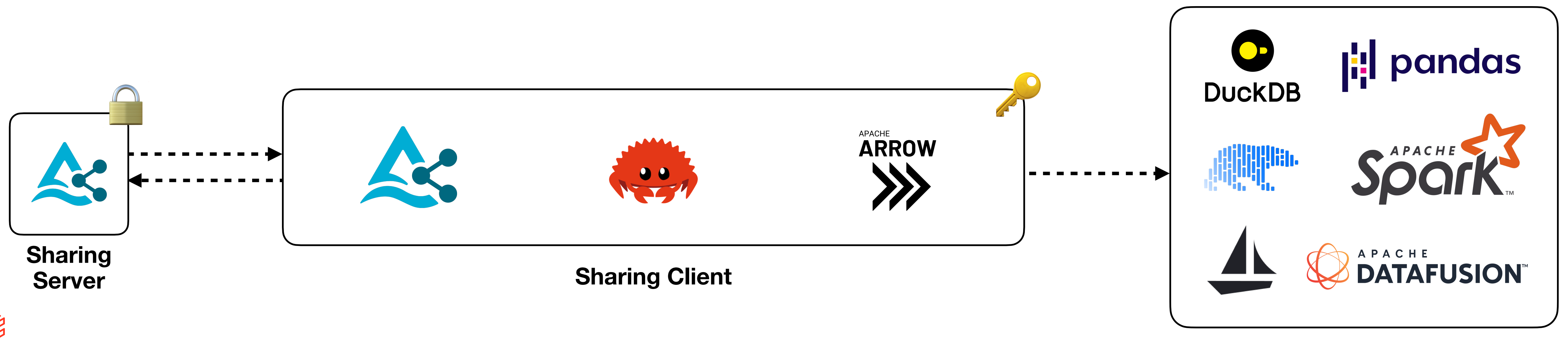
Save the metadata as a quasi delta log and retrieve via Delta Kernel as Arrow batches.

Delta Kernel

The Delta Kernel project is a set of libraries ([Java](#) and [Rust](#)) for building Delta connectors that can read from and write into Delta tables without the need to understand the [Delta protocol details](#).

You can use this library to do the following:

- Read data from small Delta tables in a single thread in a single process.
- Read data from large Delta tables using multiple threads in a single process.
- Build a complex connector for a distributed processing engine and read very large Delta tables.
- Insert data into a Delta table either from a single process or a complex distributed engine.



Example: Arrow + DuckDB



Read proposal and view in-progress PRs:
<https://github.com/delta-io/delta-sharing/issues/860>

Example: Arrow + DuckDB

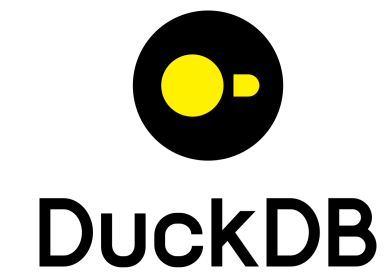


Using upcoming newer syntax*



Read proposal and view in-progress PRs:
<https://github.com/delta-io/delta-sharing/issues/860>

Example: Arrow + DuckDB



Using upcoming newer syntax*

```
import delta_sharing as ds
import duckdb
```

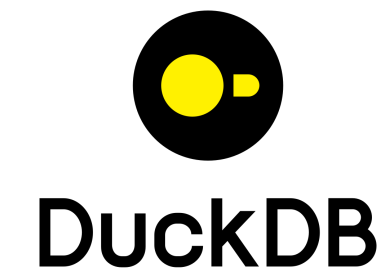
```
client = ds.SharingClient(profile="open-datasets.share")
```

```
client.list_all_tables()
```

```
[Table(name='COVID_19_NYT', share='delta_sharing', schema='default'),
 Table(name='boston-housing', share='delta_sharing', schema='default'),
 Table(name='flight-asa_2008', share='delta_sharing', schema='default'),
 Table(name='lending_club', share='delta_sharing', schema='default'),
 Table(name='nyctaxi_2019', share='delta_sharing', schema='default'),
 Table(name='nyctaxi_2019_part', share='delta_sharing', schema='default'),
 Table(name='owid-covid-data', share='delta_sharing', schema='default')]
```



Example: Arrow + DuckDB



Using upcoming newer syntax*

```
import delta_sharing as ds
import duckdb

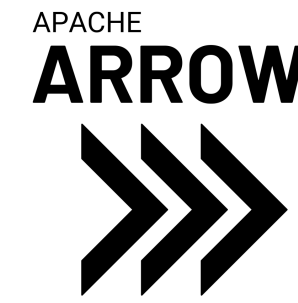
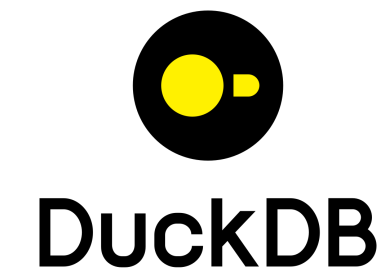
client = ds.SharingClient(profile="open-datasets.share")
client.list_all_tables()
[Table(name='COVID_19_NYT', share='delta_sharing', schema='default'),
 Table(name='boston-housing', share='delta_sharing', schema='default'),
 Table(name='flight-asa_2008', share='delta_sharing', schema='default'),
 Table(name='lending_club', share='delta_sharing', schema='default'),
 Table(name='nyctaxi_2019', share='delta_sharing', schema='default'),
 Table(name='nyctaxi_2019_part', share='delta_sharing', schema='default'),
 Table(name='owid-covid-data', share='delta_sharing', schema='default')]

table = client.table("delta_sharing.default.COVID_19_NYT")
reader = table.scan(limit=10000).to_arrow()
#reader = table.scan(limit=10000).to_record_batch_reader()

con.from_arrow(reader)
```



Example: Arrow + DuckDB



Using upcoming newer syntax*

```
import delta_sharing as ds
import duckdb

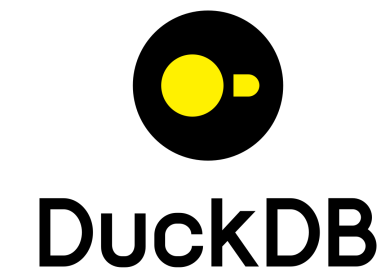
client = ds.SharingClient(profile="open-datasets.share")
client.list_all_tables()
[Table(name='COVID_19_NYT', share='delta_sharing', schema='default'),
 Table(name='boston-housing', share='delta_sharing', schema='default'),
 Table(name='flight-asa_2008', share='delta_sharing', schema='default'),
 Table(name='lending_club', share='delta_sharing', schema='default'),
 Table(name='nyctaxi_2019', share='delta_sharing', schema='default'),
 Table(name='nyctaxi_2019_part', share='delta_sharing', schema='default'),
 Table(name='owid-covid-data', share='delta_sharing', schema='default')]

table = client.table("delta_sharing.default.COVID_19_NYT")
reader = table.scan(limit=10000).to_arrow()
#reader = table.scan(limit=10000).to_record_batch_reader()

con.from_arrow(reader)
```



Example: Arrow + DuckDB



Using upcoming newer syntax*

```
import delta_sharing as ds
import duckdb

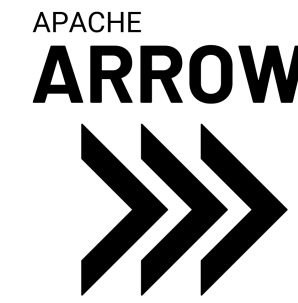
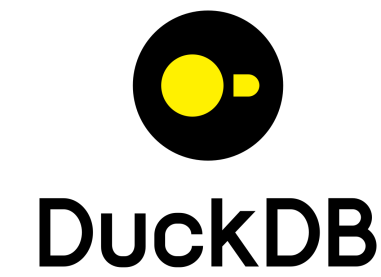
client = ds.SharingClient(profile="open-datasets.share")
client.list_all_tables()
[Table(name='COVID_19_NYT', share='delta_sharing', schema='default'),
 Table(name='boston-housing', share='delta_sharing', schema='default'),
 Table(name='flight-asa_2008', share='delta_sharing', schema='default'),
 Table(name='lending_club', share='delta_sharing', schema='default'),
 Table(name='nyctaxi_2019', share='delta_sharing', schema='default'),
 Table(name='nyctaxi_2019_part', share='delta_sharing', schema='default'),
 Table(name='owid-covid-data', share='delta_sharing', schema='default')]

table = client.table("delta_sharing.default.COVID_19_NYT")
reader = table.scan(limit=10000).to_arrow() Eager
#reader = table.scan(limit=10000).to_record_batch_reader()

con.from_arrow(reader)
```



Example: Arrow + DuckDB



Using upcoming newer syntax*

```
import delta_sharing as ds
import duckdb

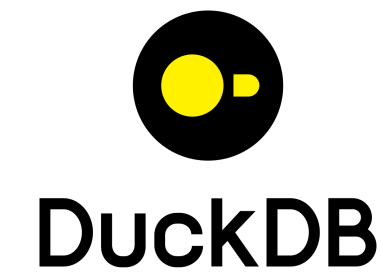
client = ds.SharingClient(profile="open-datasets.share")
client.list_all_tables()
[Table(name='COVID_19_NYT', share='delta_sharing', schema='default'),
 Table(name='boston-housing', share='delta_sharing', schema='default'),
 Table(name='flight-asa_2008', share='delta_sharing', schema='default'),
 Table(name='lending_club', share='delta_sharing', schema='default'),
 Table(name='nyctaxi_2019', share='delta_sharing', schema='default'),
 Table(name='nyctaxi_2019_part', share='delta_sharing', schema='default'),
 Table(name='owid-covid-data', share='delta_sharing', schema='default')]

table = client.table("delta_sharing.default.COVID_19_NYT")
reader = table.scan(limit=10000).to_arrow() Eager
#reader = table.scan(limit=10000).to_record_batch_reader() Lazy

con.from_arrow(reader)
```



Example: Arrow + DuckDB



Using upcoming newer syntax*

```
import delta_sharing as ds
import duckdb

client = ds.SharingClient(profile="open-datasets.share")
client.list_all_tables()
[Table(name='COVID_19_NYT', share='delta_sharing', schema='default'),
 Table(name='boston-housing', share='delta_sharing', schema='default'),
 Table(name='flight-asa_2008', share='delta_sharing', schema='default'),
 Table(name='lending_club', share='delta_sharing', schema='default'),
 Table(name='nyctaxi_2019', share='delta_sharing', schema='default'),
 Table(name='nyctaxi_2019_part', share='delta_sharing', schema='default'),
 Table(name='owid-covid-data', share='delta_sharing', schema='default')]

table = client.table("delta_sharing.default.COVID_19_NYT")
reader = table.scan(limit=10000).to_arrow() Eager
#reader = table.scan(limit=10000).to_record_batch_reader() Lazy

con.from_arrow(reader)
```




```
import delta_sharing as ds
import duckdb
```

```
client.list_all_tables()
```

```
table = client.table("delta_sharing.default.COVID_19_NYT")
reader = table.scan(limit=10000).to_arrow() Eager
#reader = table.scan(limit=10000).to_record_batch_reader() Lazy
```

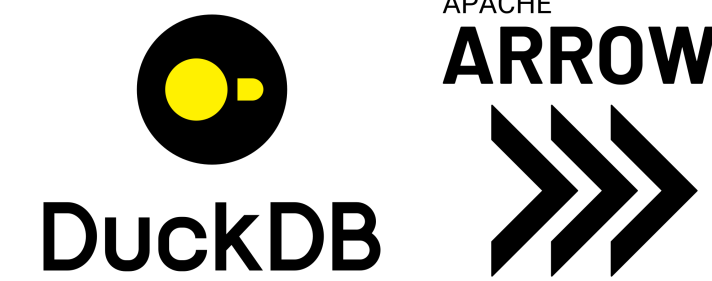
```
con.from_arrow(reader)
```

```
>>> con.from_arrow(reader)
100% ██████████ (00:00:08.71 elapsed)
```

date varchar	county varchar	state varchar	fips int32	cases int32	deaths int32
2021-01-10	Washakie	Wyoming	56043	804	21
2021-01-10	Weston	Wyoming	56045	485	4
2021-01-11	Autauga	Alabama	1001	4902	55
2021-01-11	Baldwin	Alabama	1003	15417	173
2021-01-11	Barbour	Alabama	1005	1663	35



Read proposal and view in-progress PRs:
<https://github.com/delta-io/delta-sharing/issues/860>



Using upcoming newer syntax*

```
import delta_sharing as ds
import duckdb
```

```
client = ds.SharingClient(profile="open-datasets.share")
```

```
client.list_all_tables()
```

```
[Table(name='COVID_19_NYT', share='delta_sharing', schema='default'),
 Table(name='boston-housing', share='delta_sharing', schema='default'),
 Table(name='flight-asa_2008', share='delta_sharing', schema='default'),
 Table(name='lending_club', share='delta_sharing', schema='default'),
 Table(name='nyctaxi_2019', share='delta_sharing', schema='default'),
 Table(name='nyctaxi_2019_part', share='delta_sharing', schema='default'),
 Table(name='owid-covid-data', share='delta_sharing', schema='default')]
```

```
table = client.table("delta_sharing.default.COVID_19_NYT")
```

```
reader = table.scan(limit=10000).to_arrow() Eager
```

```
#reader = table.scan(limit=10000).to_record_batch_reader() Lazy
```

```
con.from_arrow(reader)
```

- **Simplified user experienced**

```
>>> con.from_arrow(reader)
```

100% ██████████ (00:00:08.71 elapsed)

date varchar	county varchar	state varchar	fips int32	cases int32	deaths int32
2021-01-10	Washakie	Wyoming	56043	804	21
2021-01-10	Weston	Wyoming	56045	485	4
2021-01-11	Autauga	Alabama	1001	4902	55
2021-01-11	Baldwin	Alabama	1003	15417	173
2021-01-11	Barbour	Alabama	1005	1663	35



Read proposal and view in-progress PRs:
<https://github.com/delta-io/delta-sharing/issues/860>


```
import delta_sharing as ds
import duckdb
```

```
client.list_all_tables()
```

```
[Table(name='COVID_19_NYT', share='delta_sharing', schema='default'),
 Table(name='boston-housing', share='delta_sharing', schema='default'),
 Table(name='flight-asa_2008', share='delta_sharing', schema='default'),
 Table(name='lending_club', share='delta_sharing', schema='default'),
 Table(name='nyctaxi_2019', share='delta_sharing', schema='default'),
 Table(name='nyctaxi_2019_part', share='delta_sharing', schema='default'),
 Table(name='owid-covid-data', share='delta_sharing', schema='default')]
```

```
reader = table.scan(limit=10000).to_arrow() Eager
```

```
#reader = table.scan(limit=10000).to_record_batch_reader() Lazy
```

```
con.from_arrow(reader)
```

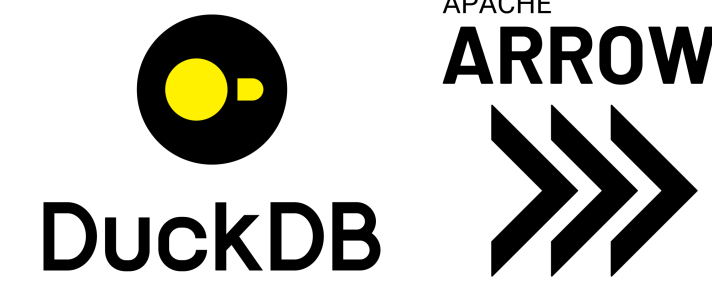
- Simplified user experienced
- Improved interoperability via Arrow

```
>>> con.from_arrow(reader)
100% ██████████ (00:00:08.71 elapsed)
```

date varchar	county varchar	state varchar	fips int32	cases int32	deaths int32
2021-01-10	Washakie	Wyoming	56043	804	21
2021-01-10	Weston	Wyoming	56045	485	4
2021-01-11	Autauga	Alabama	1001	4902	55
2021-01-11	Baldwin	Alabama	1003	15417	173
2021-01-11	Barbour	Alabama	1005	1663	35



Read proposal and view in-progress PRs:
<https://github.com/delta-io/delta-sharing/issues/860>



Using upcoming newer syntax*

```
import delta_sharing as ds
import duckdb
```

```
client = ds.SharingClient(profile="open-datasets.share")
```

```
client.list_all_tables()
```

```
[Table(name='COVID_19_NYT', share='delta_sharing', schema='default'),
 Table(name='boston-housing', share='delta_sharing', schema='default'),
 Table(name='flight-asa_2008', share='delta_sharing', schema='default'),
 Table(name='lending_club', share='delta_sharing', schema='default'),
 Table(name='nyctaxi_2019', share='delta_sharing', schema='default'),
 Table(name='nyctaxi_2019_part', share='delta_sharing', schema='default'),
 Table(name='owid-covid-data', share='delta_sharing', schema='default')]
```

```
table = client.table("delta_sharing.default.COVID_19_NYT")
```

```
reader = table.scan(limit=10000).to_arrow() Eager
```

```
#reader = table.scan(limit=10000).to_record_batch_reader() Lazy
```

```
con.from_arrow(reader)
```

- Simplified user experienced
- Improved interoperability via *Arrow*
- Less overhead, higher throughput

```
>>> con.from_arrow(reader)
```

100% ██████████ (00:00:08.71 elapsed)

date varchar	county varchar	state varchar	fips int32	cases int32	deaths int32
2021-01-10	Washakie	Wyoming	56043	804	21
2021-01-10	Weston	Wyoming	56045	485	4
2021-01-11	Autauga	Alabama	1001	4902	55
2021-01-11	Baldwin	Alabama	1003	15417	173
2021-01-11	Barbour	Alabama	1005	1663	35



Read proposal and view in-progress PRs:
<https://github.com/delta-io/delta-sharing/issues/860>

Performance

Using upcoming newer syntax*



Read proposal and view in-progress PRs:
<https://github.com/delta-io/delta-sharing/issues/860>

Performance

Using upcoming newer syntax*

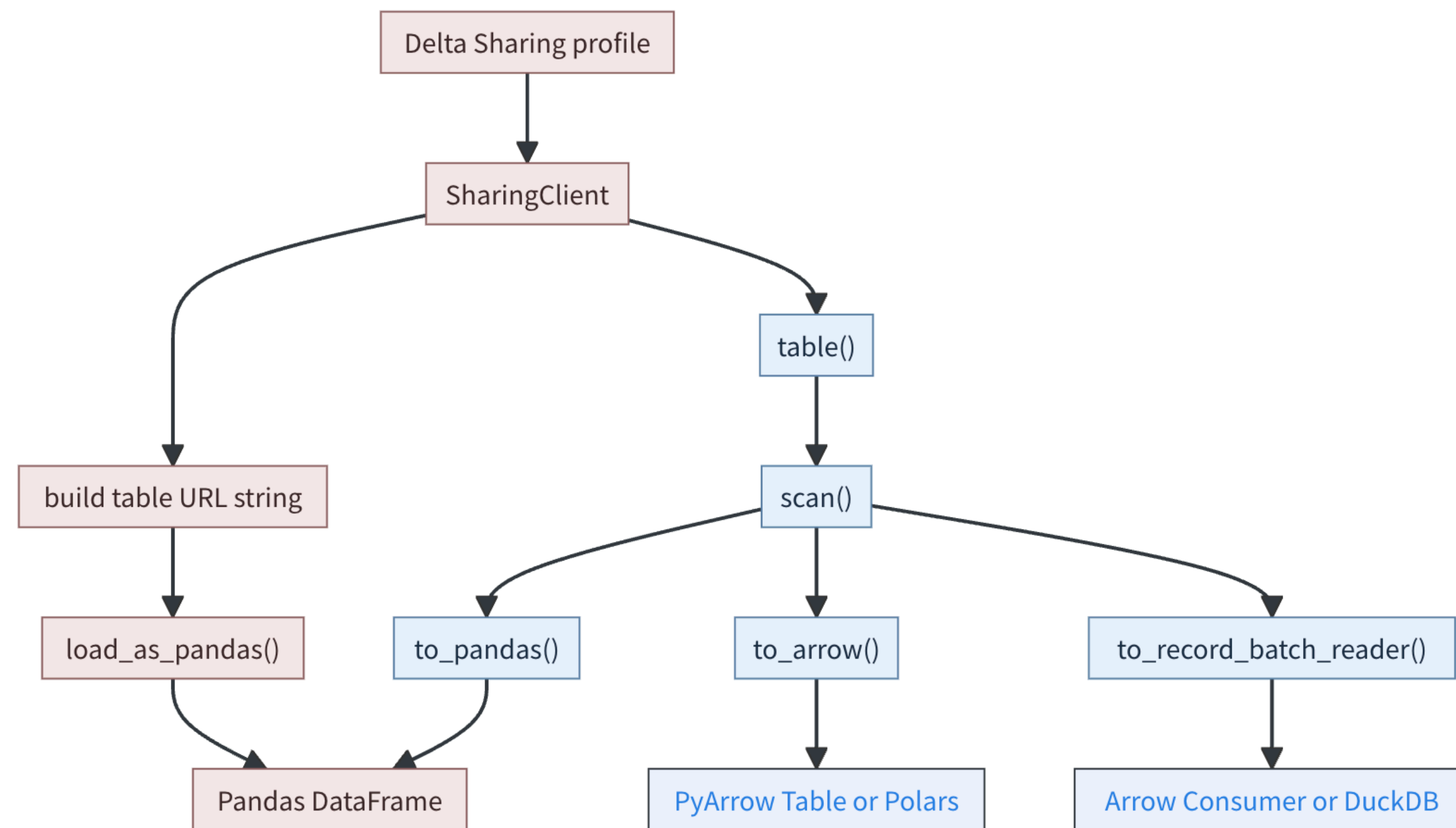


Figure 1: The red path shows the earlier public interface. The blue path shows the newer table-and-scan interface, which branches to Arrow-native outputs.



Performance

Using upcoming newer syntax*

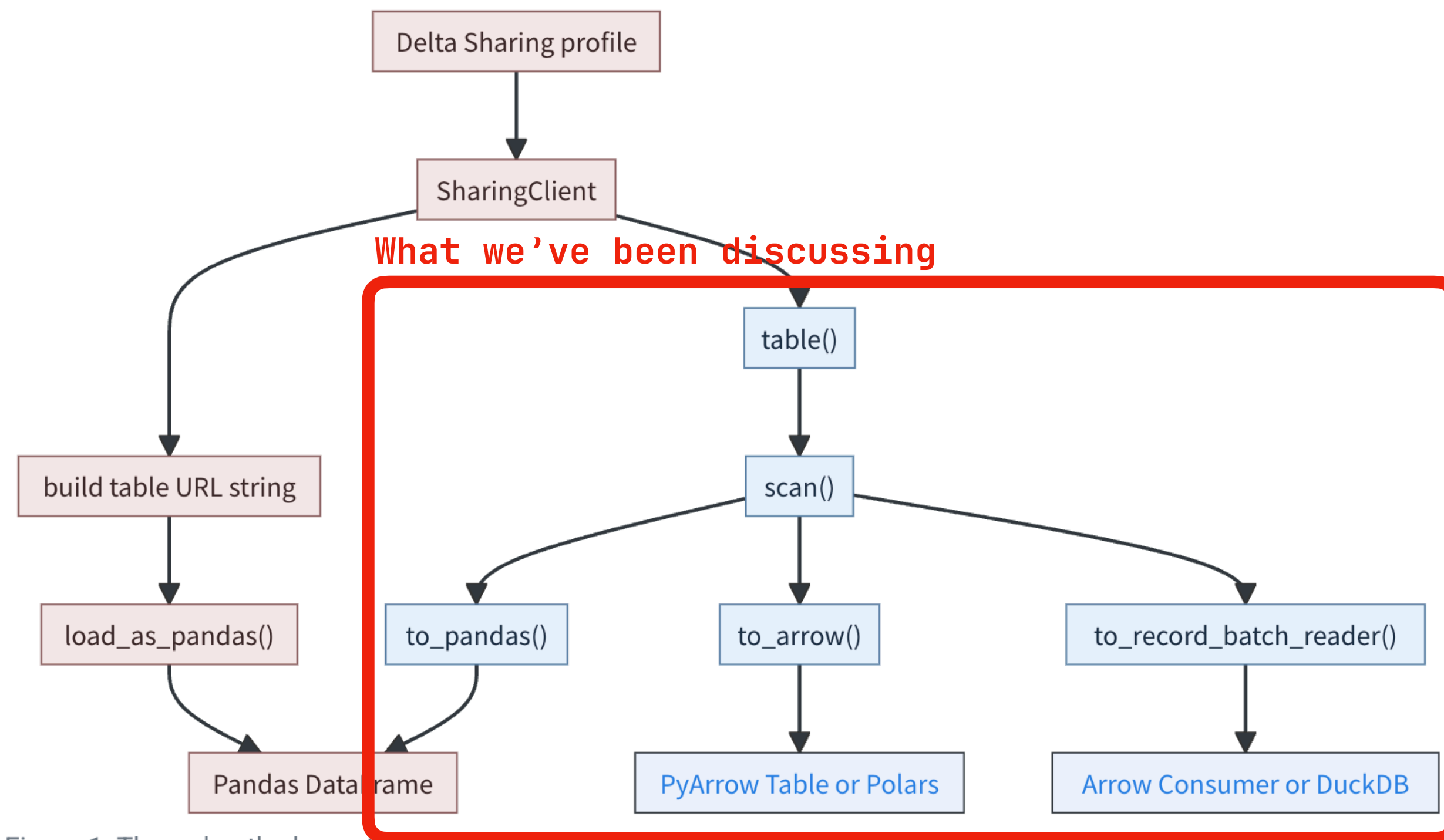


Figure 1: The red path shows the earlier public interface. The blue path shows the newer table-and-scan interface, which branches to Arrow-native outputs.



Performance

Using upcoming newer syntax*

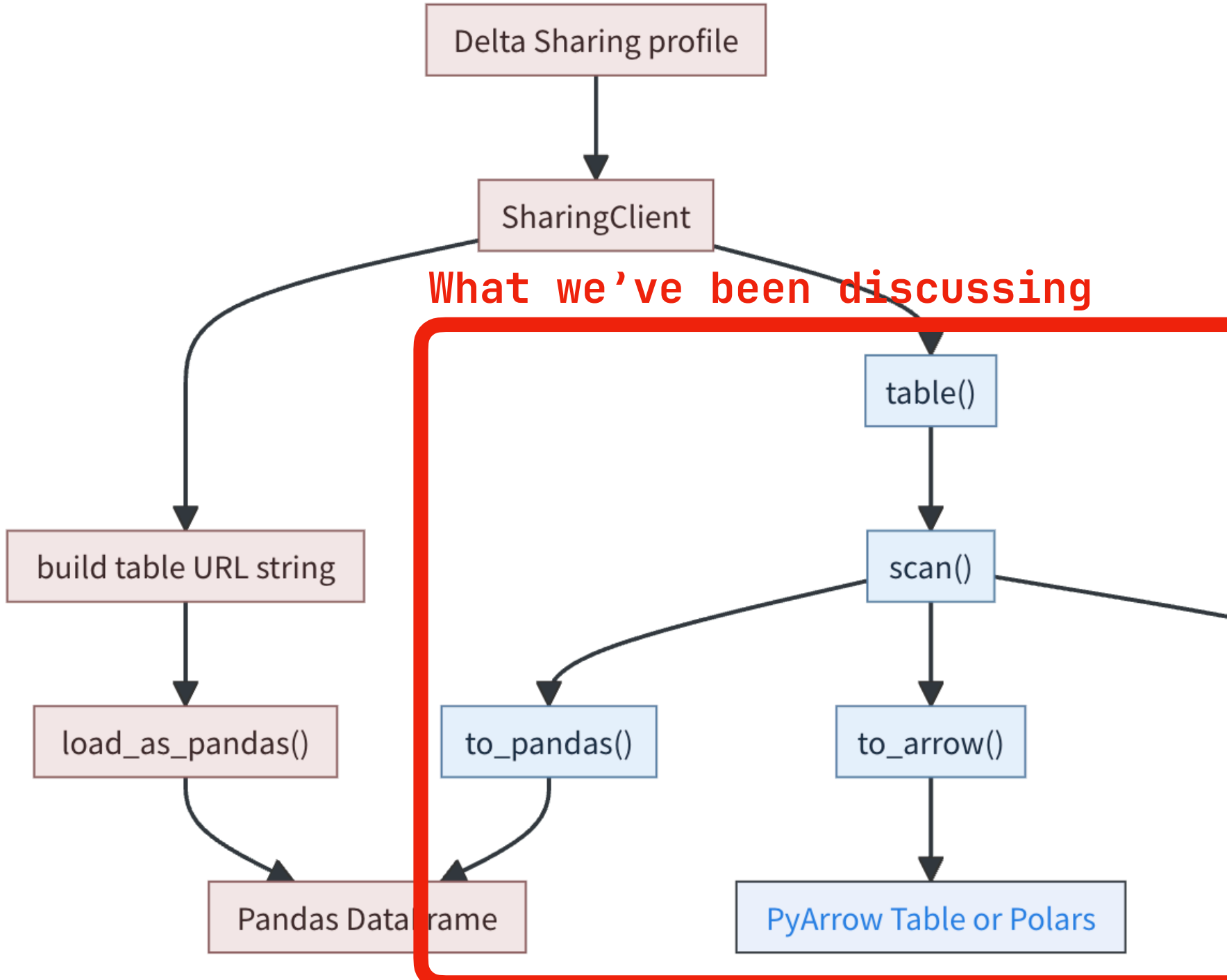


Figure 1: The red path shows the earlier public interface. The blue path shows the new which branches to Arrow-native outputs.

First, does it actually run faster?

► Expand the runtime benchmark script

Runtime benchmark results for 10 million rows from the Delta Sharing open-share example. Lower is better.

Method	Mean runtime (s)	Min (s)	Max (s)
<code>load_as_pandas(...)</code>	59.08	56.47	70.42
<code>scan.to_pandas()</code>	60.06	56.59	80.86
 <code>scan.to_arrow()</code>	21.63	20.93	23.41
DuckDB + <code>scan.to_record_batch_reader()</code>	23.20	21.75	24.37

At 10 million rows, the two Pandas paths are effectively the same at about 60 seconds. The Arrow-native paths are substantially faster, reducing mean runtime by roughly 60% relative to the Pandas reads.

Performance

Using upcoming newer syntax*

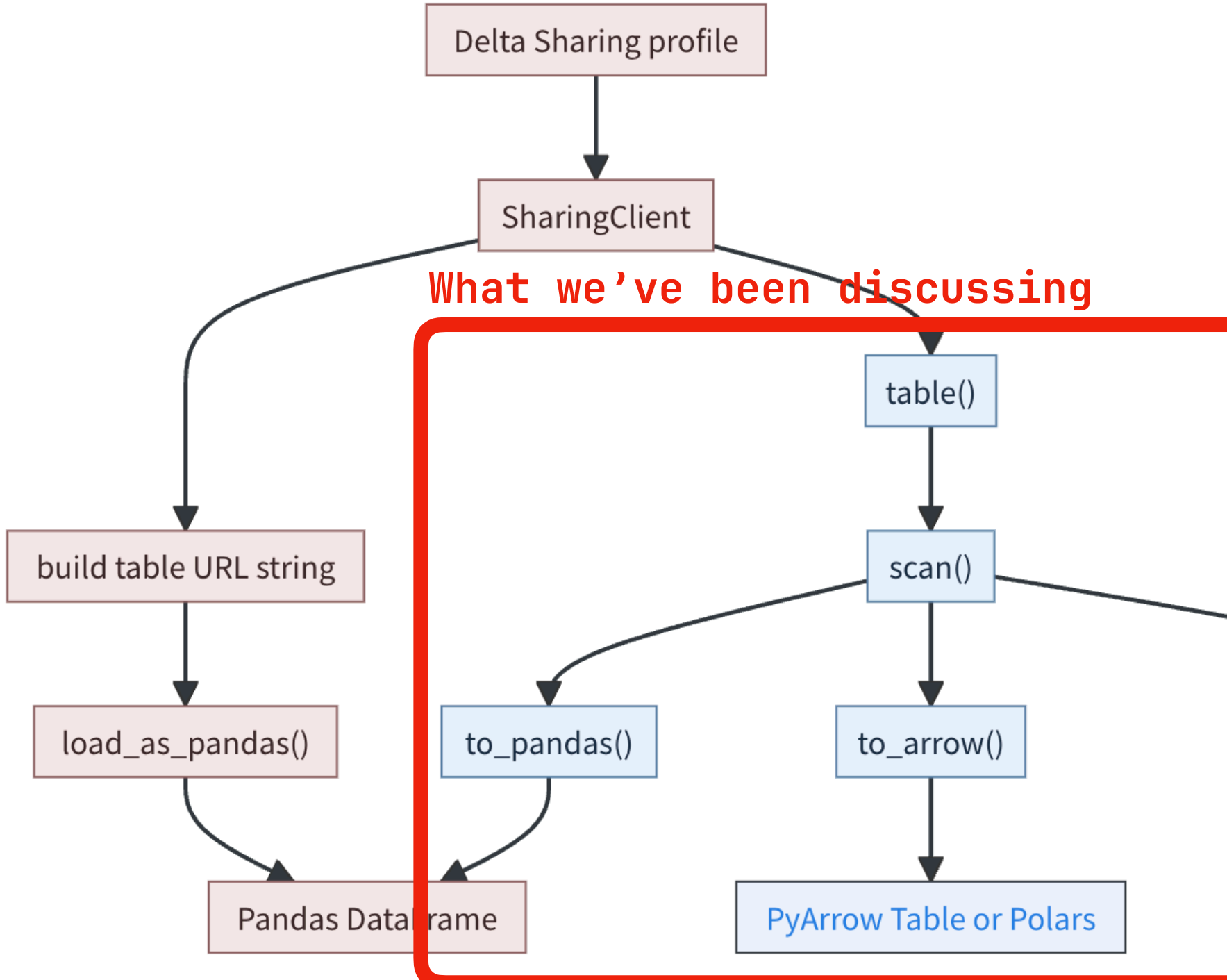


Figure 1: The red path shows the earlier public interface. The blue path shows the new which branches to Arrow-native outputs.

First, does it actually run faster?

► Expand the runtime benchmark script

Runtime benchmark results for 10 million rows from the Delta Sharing open-share example. Lower is better.

Method	Mean runtime (s)	Min (s)	Max (s)
<code>load_as_pandas(...)</code>	59.08	56.47	70.42
<code>scan.to_pandas()</code>	60.06	56.59	80.86
<code>scan.to_arrow()</code>	21.63	20.93	23.41
DuckDB + <code>scan.to_record_batch_reader()</code>	23.20	21.75	24.37

~2.7x

At 10 million rows, the two Pandas paths are effectively the same at about 60 seconds. The Arrow-native paths are substantially faster, reducing mean runtime by roughly 60% relative to the Pandas reads.

Performance

Using upcoming newer syntax*

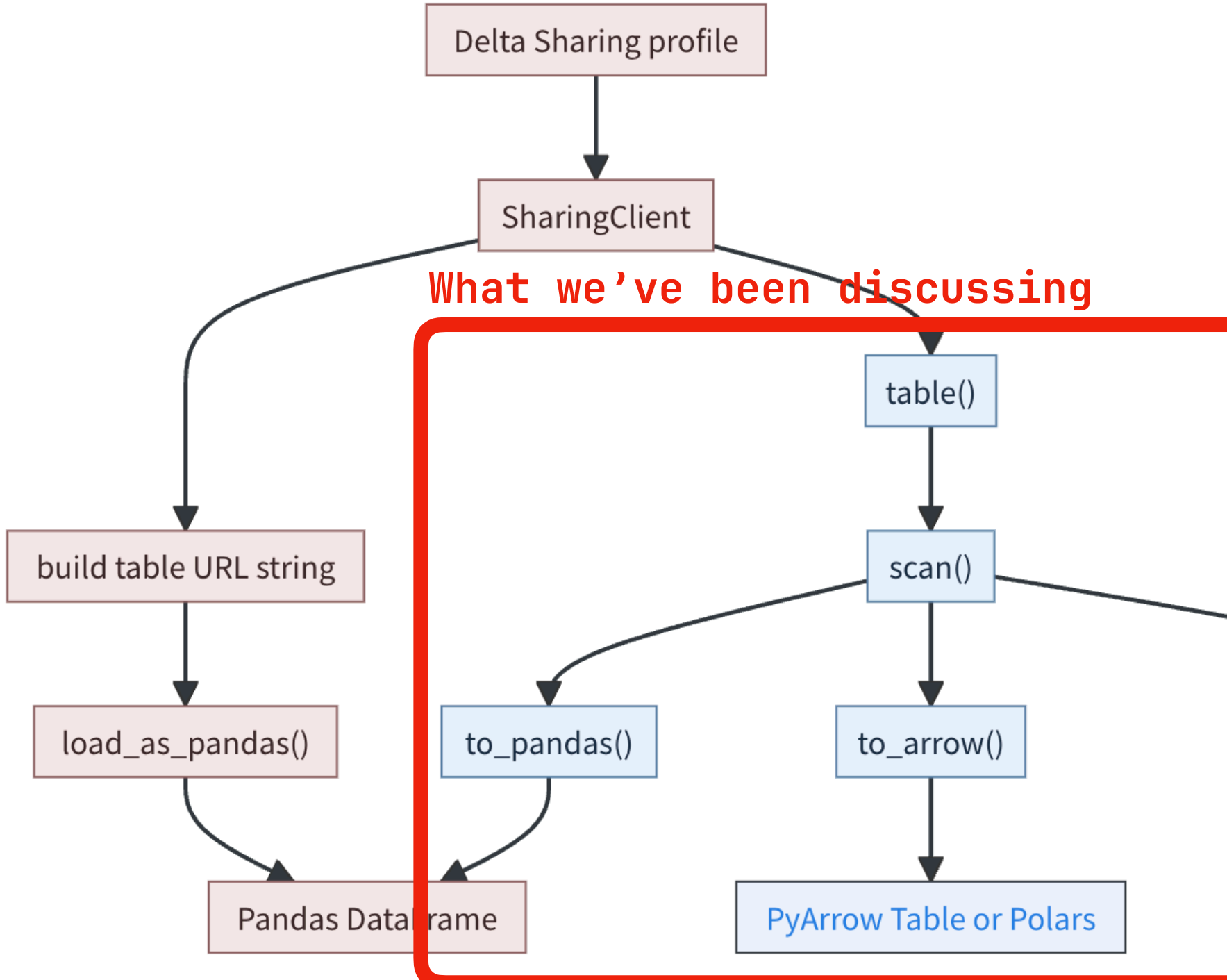


Figure 1: The red path shows the earlier public interface. The blue path shows the new which branches to Arrow-native outputs.

Then, what happens when you stop reading it all at once?

► Expand the peak memory benchmark script

Measured as the maximum memory used during the run.

Memory benchmark results for incremental batch writes versus eager Arrow materialization and write. Lower is better.

Method	Runtime mean (s)	Peak mean (MiB)	Peak max (MiB)
 Incremental write from <code>scan.to_record_batch_reader()</code>	31.94	1310.63	1335.25
Read all via <code>scan.to_arrow()</code> then write	31.36	1945.57	1966.86

Mean runtime is similar in both cases, but peak memory is not. Incremental writes from `scan.to_record_batch_reader()` reduce mean peak memory by ~33% relative to reading the full Arrow table and writing afterwards.

Performance

Using upcoming newer syntax*

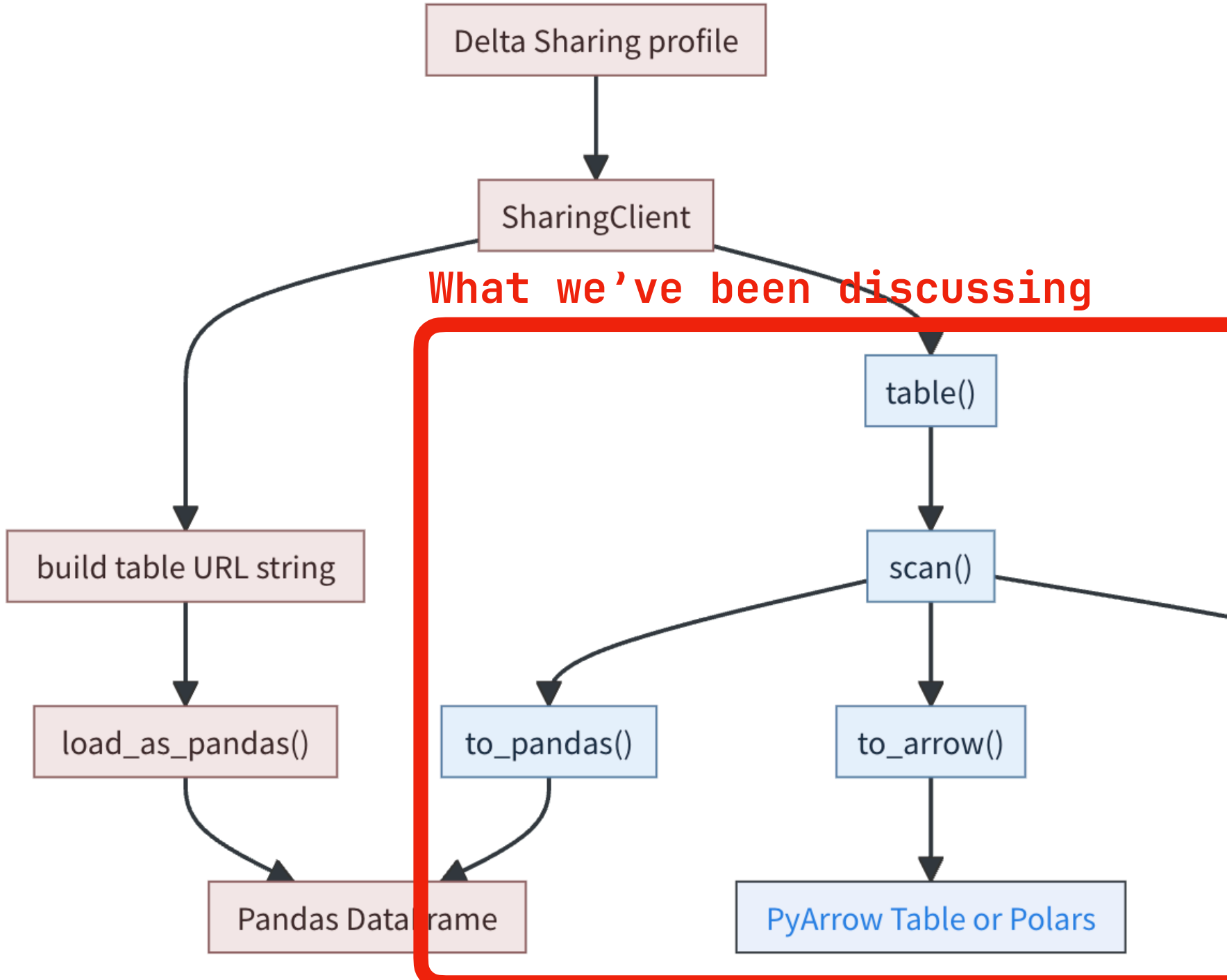


Figure 1: The red path shows the earlier public interface. The blue path shows the new which branches to Arrow-native outputs.

Then, what happens when you stop reading it all at once?

- Expand the peak memory benchmark script

Measured as the maximum memory used during the run.

Memory benchmark results for incremental batch writes versus eager Arrow materialization and write. Lower is better.

Method	Runtime mean (s)	Peak mean (MiB)	Peak max (MiB)
 Incremental write from <code>scan.to_record_batch_reader()</code>	31.94	1310.63	1335.25
Read all via <code>scan.to_arrow()</code> then write	31.36	1945.57	1966.86

Mean runtime is similar in both cases, but peak memory is not. Incremental writes from `scan.to_record_batch_reader()` reduce mean peak memory by ~33% relative to reading the full Arrow table and writing afterwards.

Performance

Using upcoming newer syntax*

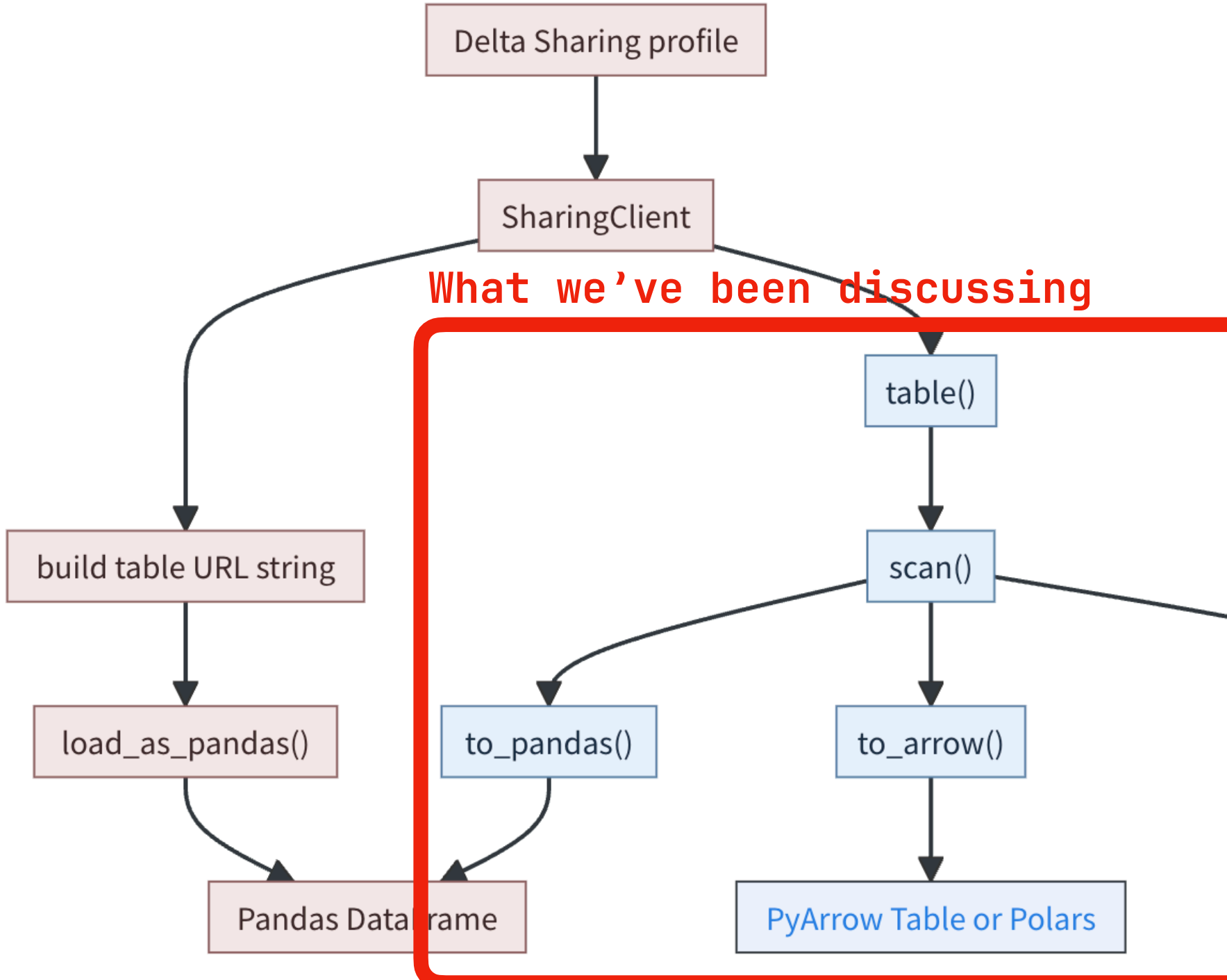


Figure 1: The red path shows the earlier public interface. The blue path shows the new which branches to Arrow-native outputs.

Then, what happens when you stop reading it all at once?

► Expand the peak memory benchmark script

Measured as the maximum memory used during the run.

Memory benchmark results for incremental batch writes versus eager Arrow materialization and write. Lower is better.

Method	Runtime mean (s)	Peak mean (MiB)	Peak max (MiB)
🏆 Incremental write from <code>scan.to_record_batch_reader()</code>	31.94	1310.63	1335.25
Read all via <code>scan.to_arrow()</code> then write	31.36	1945.57	1966.86

~33% less peak memory

Mean runtime is similar in both cases, but peak memory is not. Incremental writes from `scan.to_record_batch_reader()` reduce mean peak memory by ~33% relative to reading the full Arrow table and writing afterwards.

Views?

Data Provider



Col1	Col2	Col3	...
X	1	{}	...
X	2	{1,2}	...
Y	3	{}	...
Y	4	{10}	...

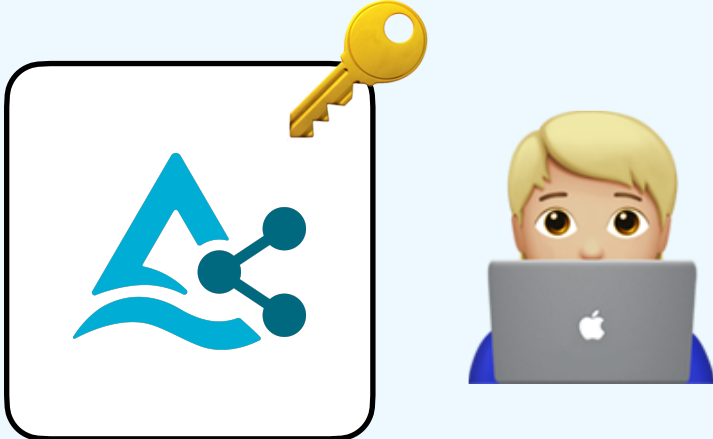


GCS / S3 / ADLS



Sharing Server

Data Recipient

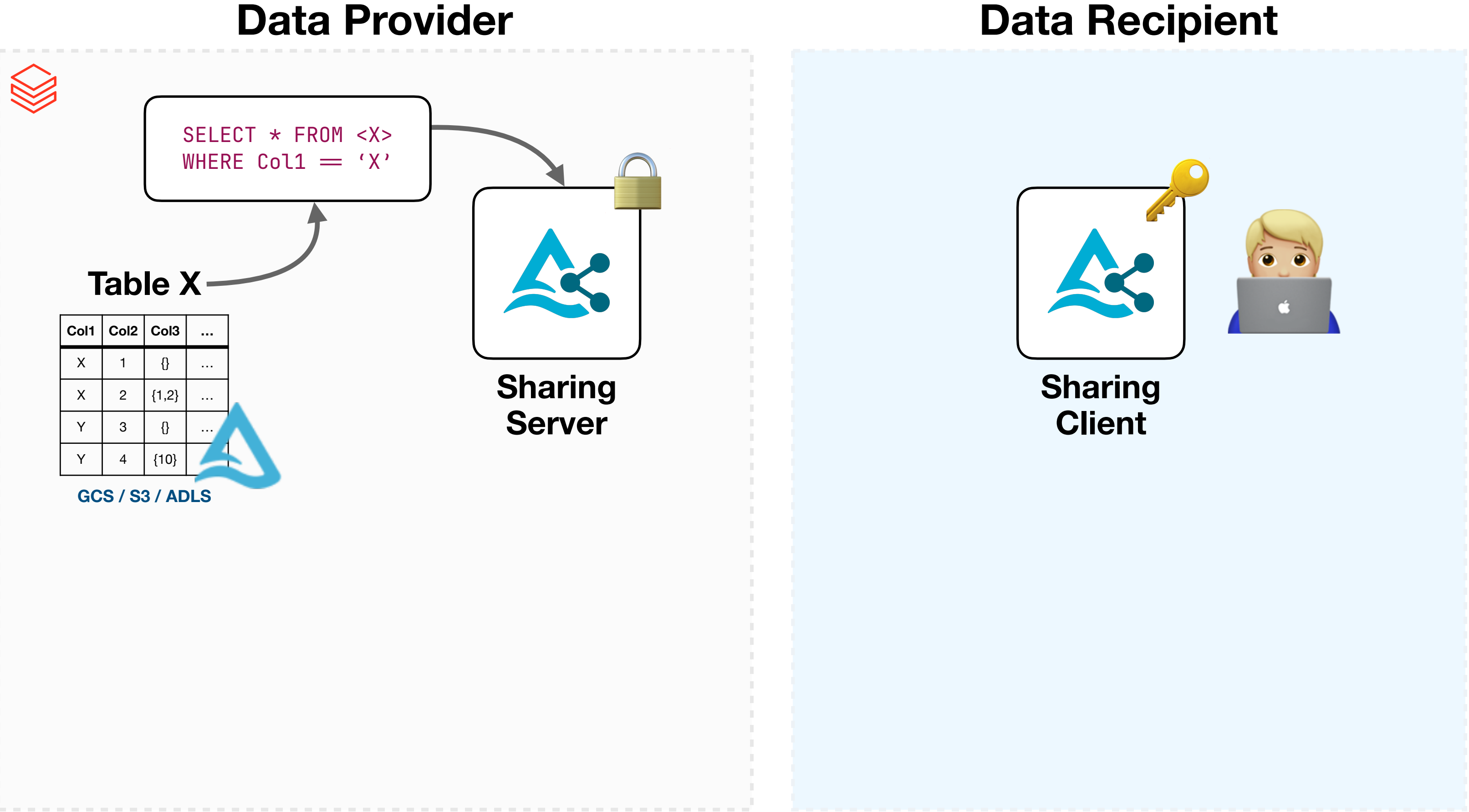


Sharing Client



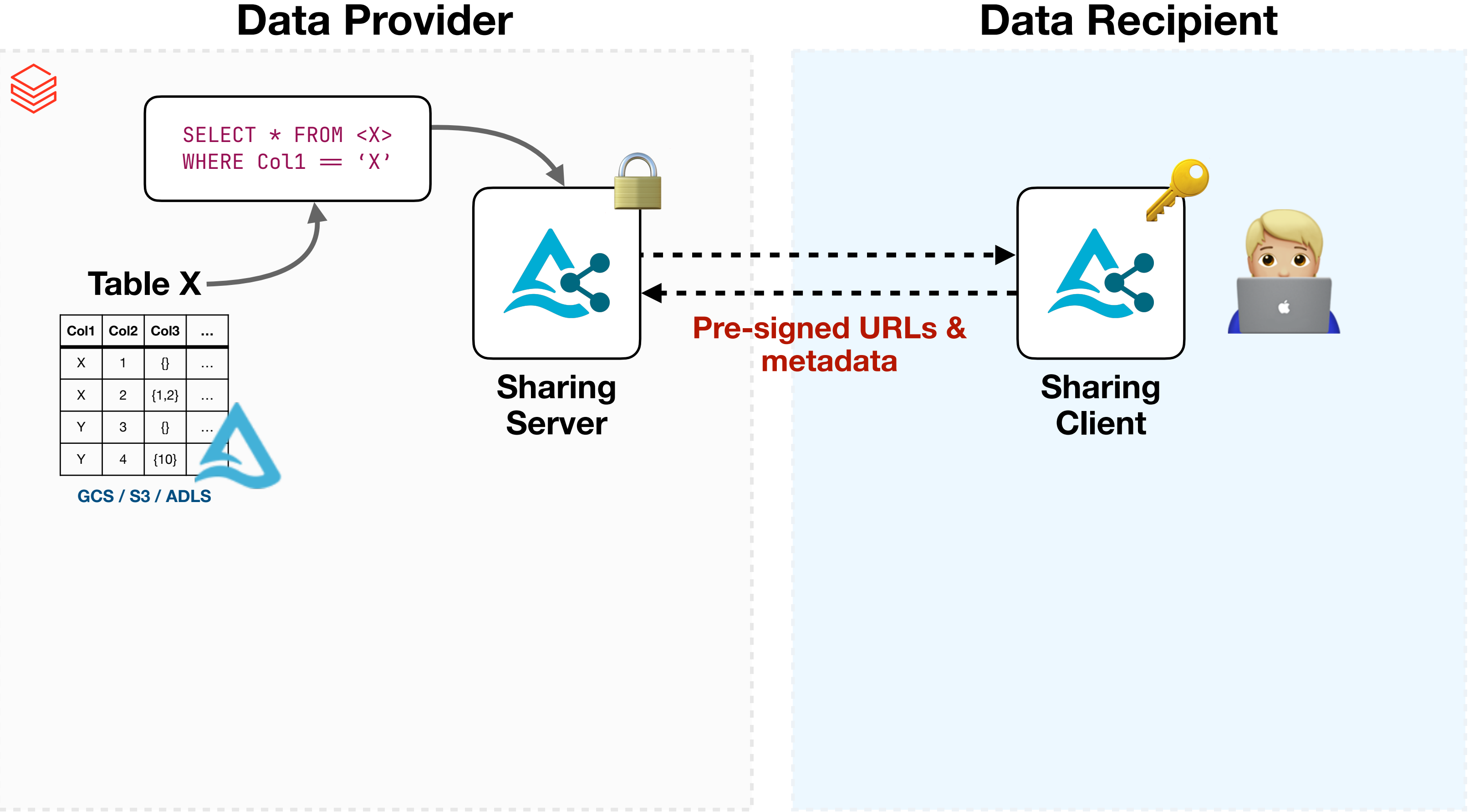
*<https://docs.databricks.com/aws/en/delta-sharing/#accessing-views>

Views?



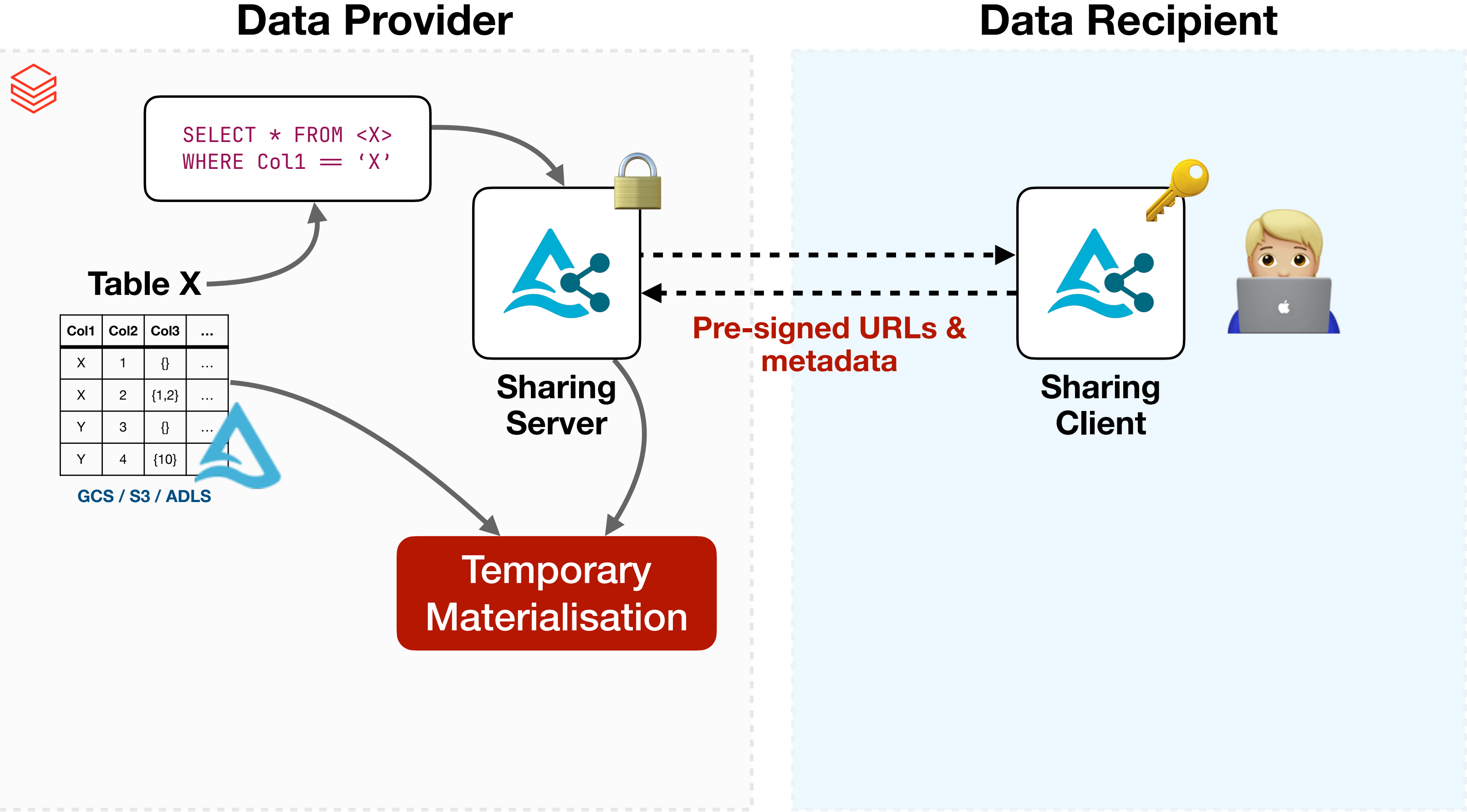
*<https://docs.databricks.com/aws/en/delta-sharing/#accessing-views>

Views?



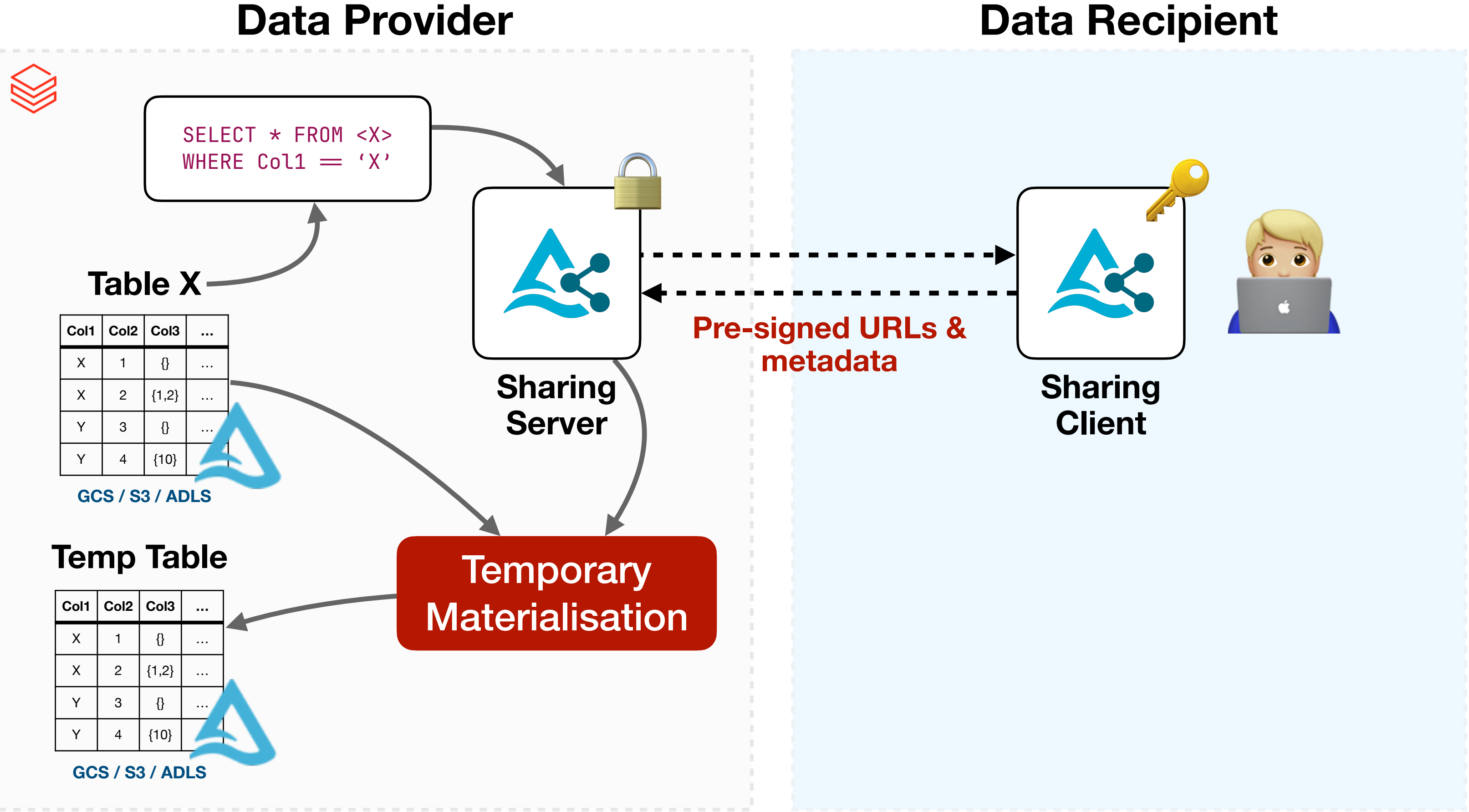
*<https://docs.databricks.com/aws/en/delta-sharing/#accessing-views>

Views?



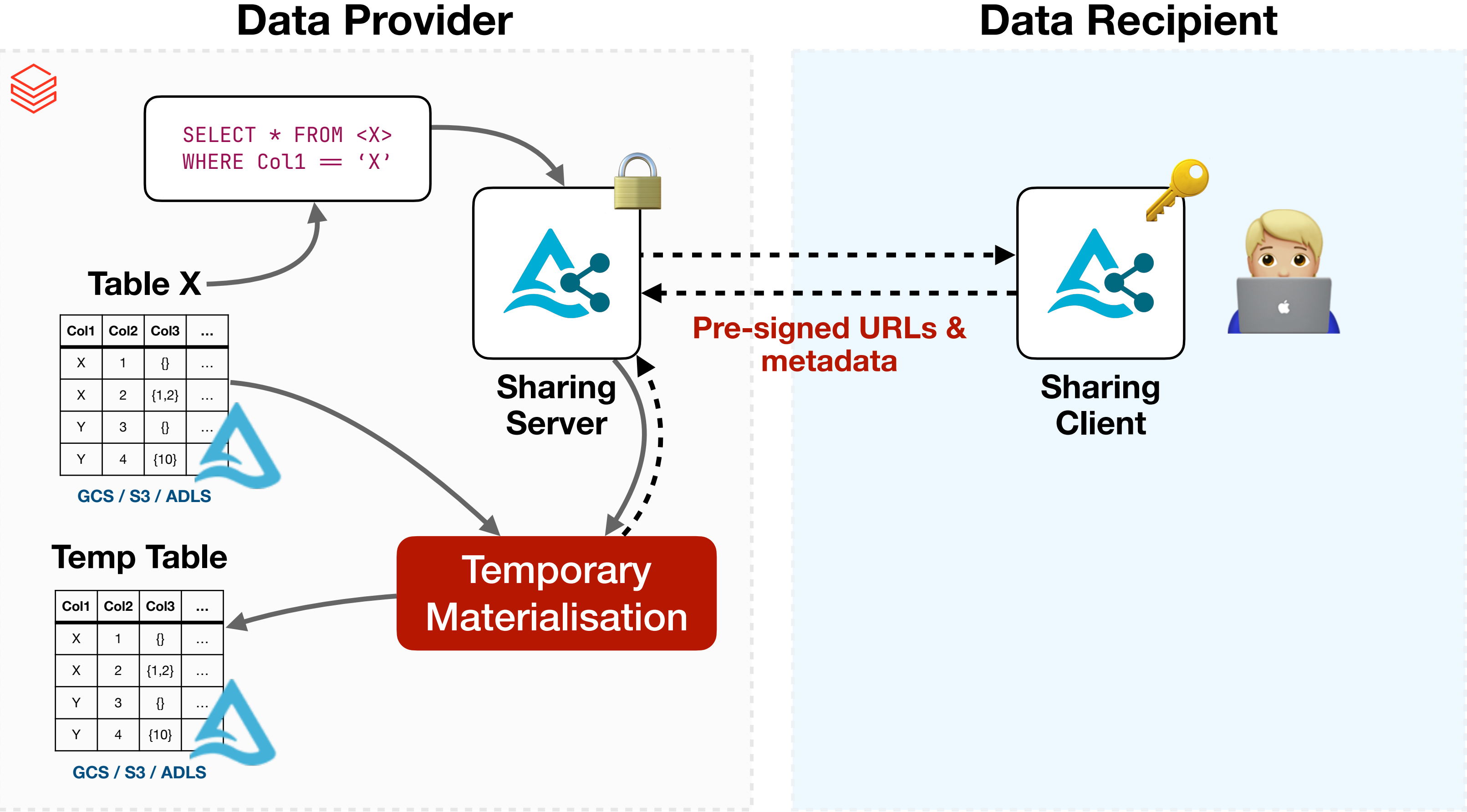
*<https://docs.databricks.com/aws/en/delta-sharing/#accessing-views>

Views?



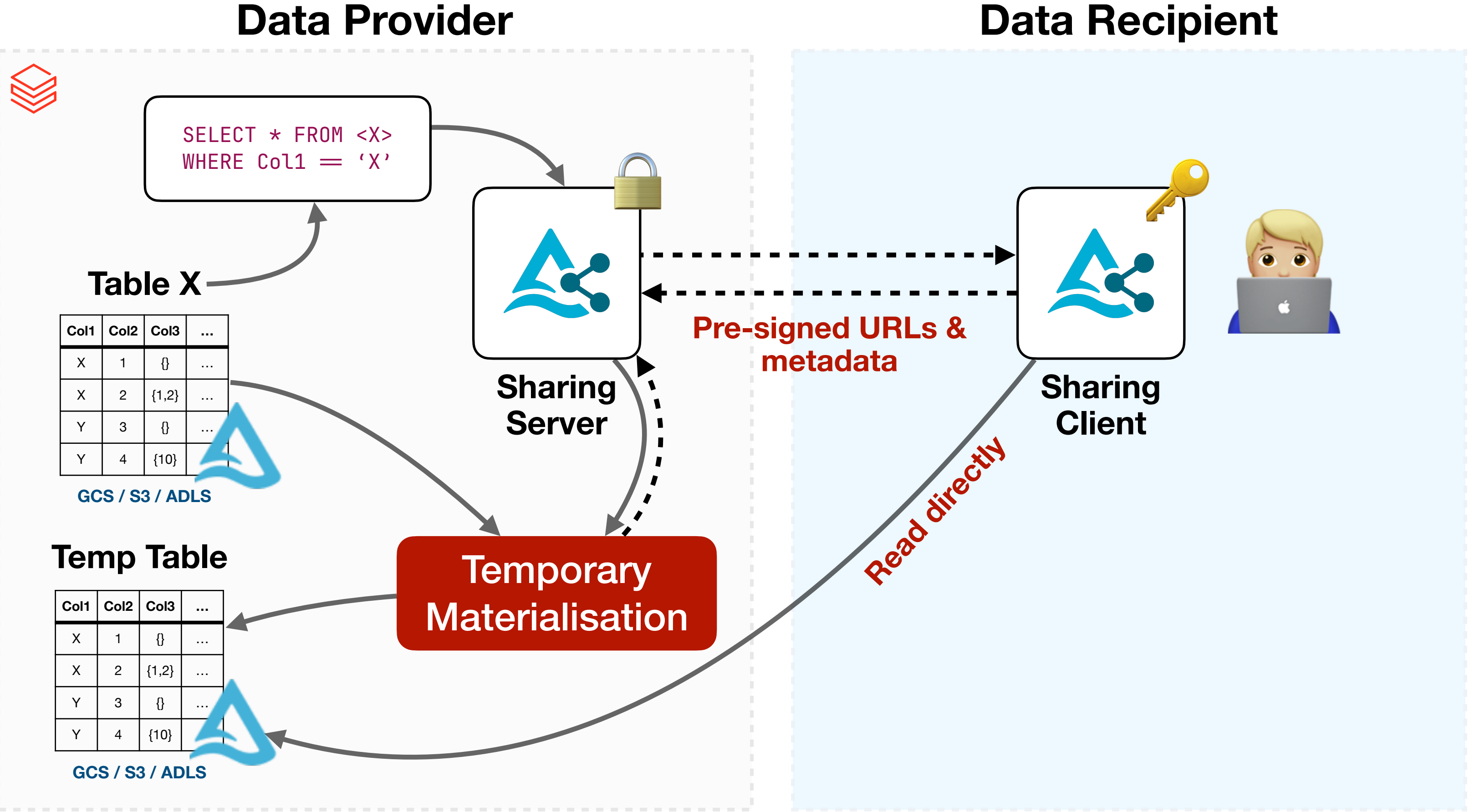
*<https://docs.databricks.com/aws/en/delta-sharing/#accessing-views>

Views?



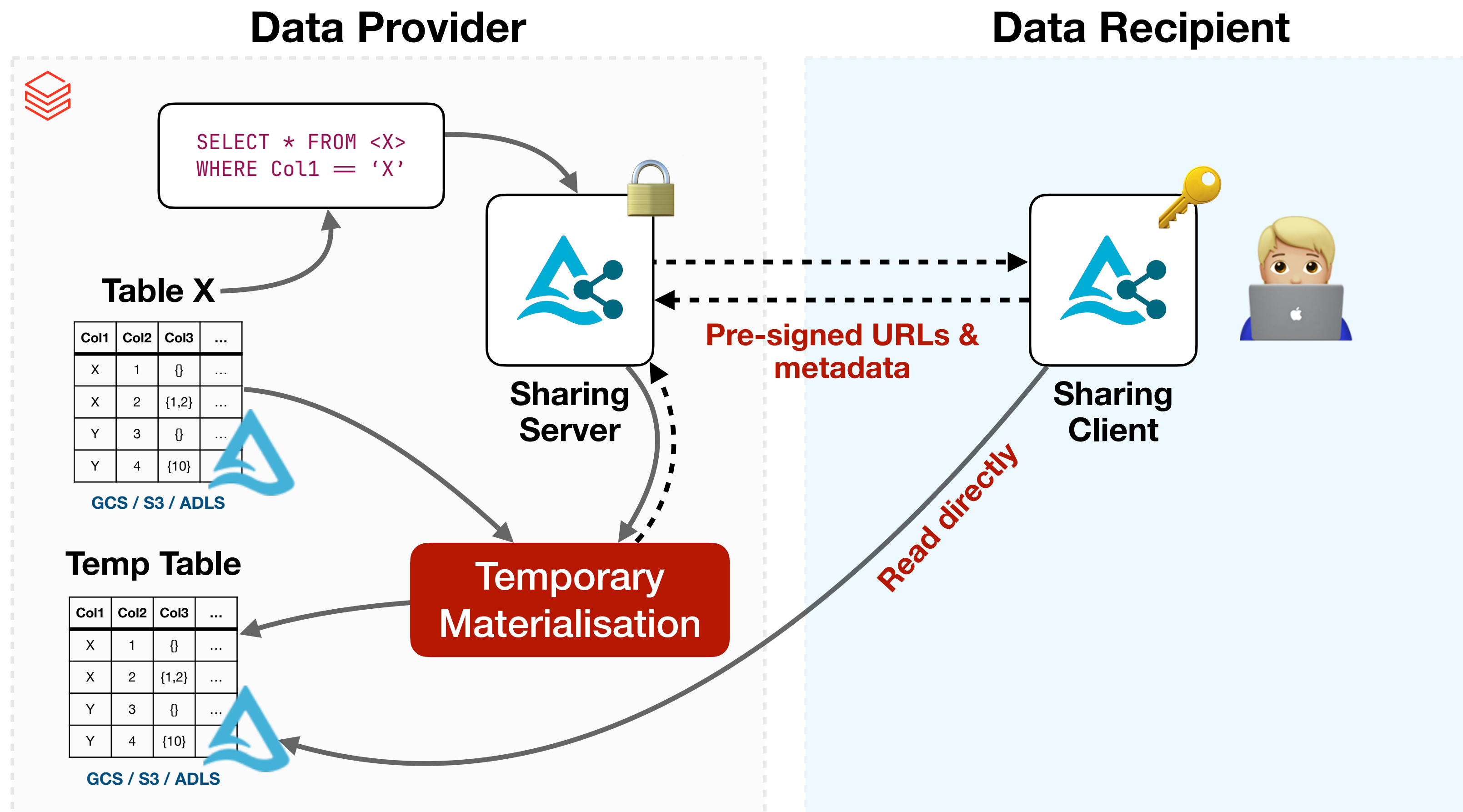
*<https://docs.databricks.com/aws/en/delta-sharing/#accessing-views>

Views?



*<https://docs.databricks.com/aws/en/delta-sharing/#accessing-views>

Views?

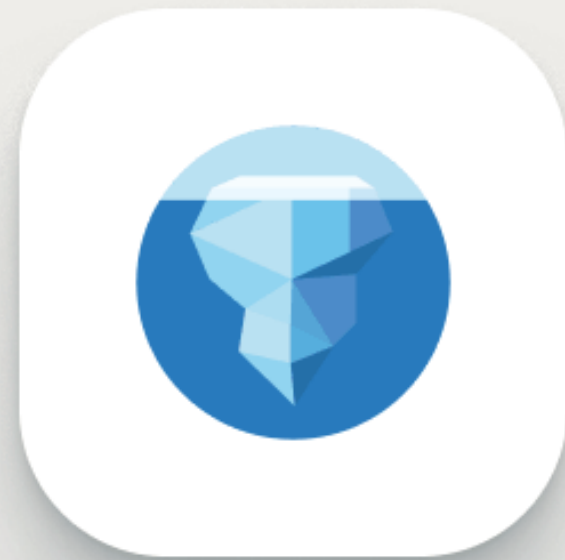


- Only supported when provider is sharing from Databricks
- Views defer computation until query time
- Sharing views requires **provider side materialisation** to be secure*
- After materialisation pre-signing still occur

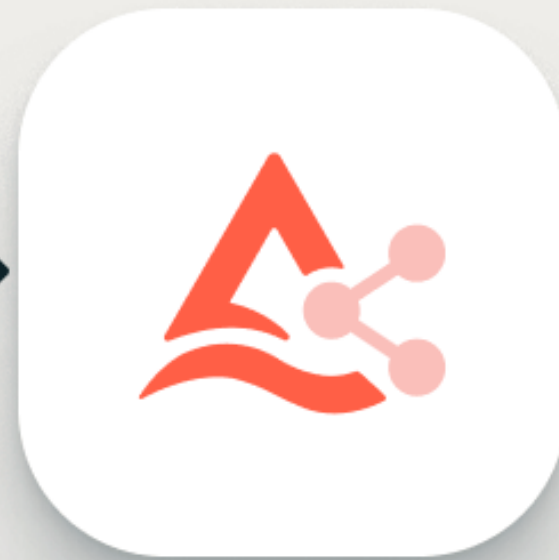
*<https://docs.databricks.com/aws/en/delta-sharing/#accessing-views>

Iceberg?

**UC Iceberg
Tables**



**Delta Sharing
Ecosystem**

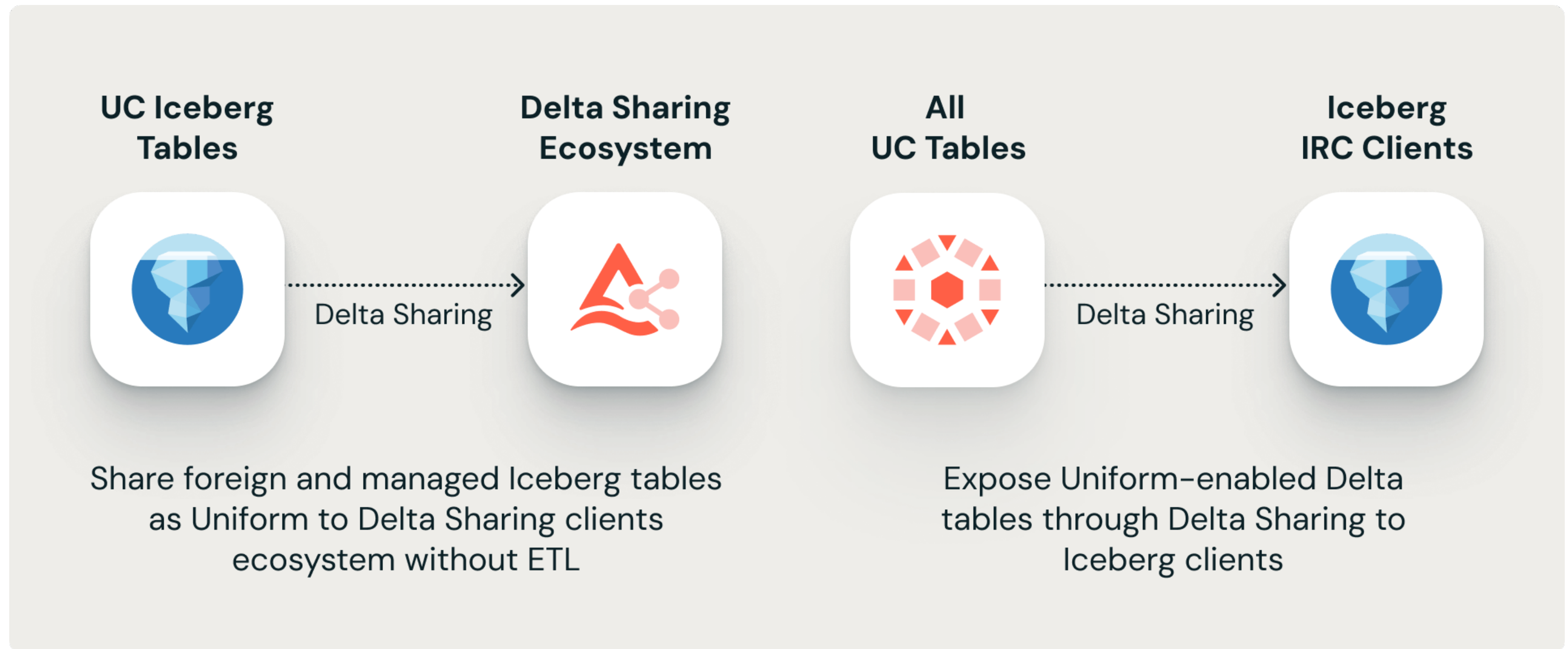


.....→
Delta Sharing

Share foreign and managed Iceberg tables
as Uniform to Delta Sharing clients
ecosystem without ETL



Iceberg?



Iceberg?

Universal Format (UniForm)

Delta Universal Format (UniForm) allows you to read Delta tables with Iceberg and Hudi clients.

UniForm takes advantage of the fact that Delta Lake, Iceberg, and Hudi all consist of Parquet data files and a metadata layer. UniForm automatically generates Iceberg metadata asynchronously, allowing Iceberg clients to read Delta tables as if they were Iceberg or Hudi tables. You can expect negligible Delta write overhead when UniForm is enabled, as the metadata conversion and transaction occurs asynchronously after the Delta commit.

A single copy of the data files provides access to clients of all formats.



Iceberg?

Universal Format (UniForm)

Delta Universal Format (UniForm) allows you to read Delta tables with Iceberg and Hudi clients.

UniForm takes advantage of the fact that Delta Lake, Iceberg, and Hudi all consist of Parquet data files and a metadata layer. UniForm automatically generates Iceberg metadata asynchronously, allowing Iceberg clients to read Delta tables as if they were Iceberg or Hudi tables. You can expect negligible Delta write overhead when UniForm is enabled, as the metadata conversion and transaction occurs asynchronously after the Delta commit.

A single copy of the data files provides access to clients of all formats.



Iceberg?

Universal Format (UniForm)

Delta Universal Format (UniForm) allows you to read Delta tables with Iceberg and Hudi clients.

UniForm takes advantage of the fact that Delta Lake, Iceberg, and Hudi all consist of Parquet data files and a metadata layer. UniForm automatically generates Iceberg metadata asynchronously, allowing Iceberg clients to read Delta tables as if they were Iceberg or Hudi tables. You can expect negligible Delta write overhead when UniForm is enabled, as the metadata conversion and transaction occurs asynchronously after the Delta commit.

A single copy of the data files provides access to clients of all formats.



Iceberg?

Universal Format (UniForm)

Delta Universal Format (UniForm) allows you to read Delta tables with Iceberg and Hudi clients.

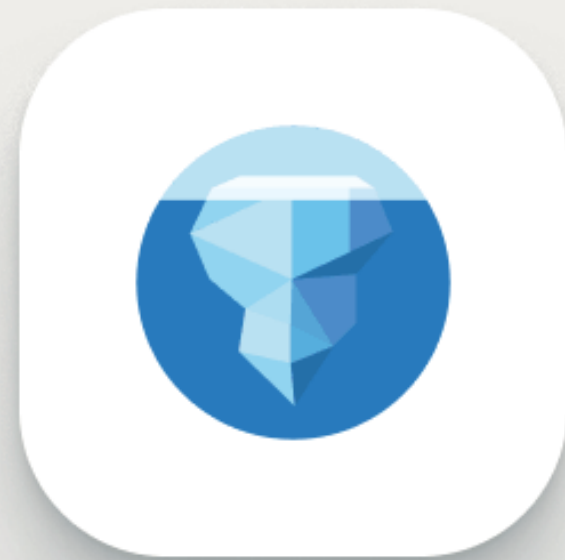
UniForm takes advantage of the fact that Delta Lake, Iceberg, and Hudi all consist of Parquet data files and a metadata layer. UniForm automatically generates Iceberg metadata asynchronously, allowing Iceberg clients to read Delta tables as if they were Iceberg or Hudi tables. You can expect negligible Delta write overhead when UniForm is enabled, as the metadata conversion and transaction occurs asynchronously after the Delta commit.

A single copy of the data files provides access to clients of all formats.



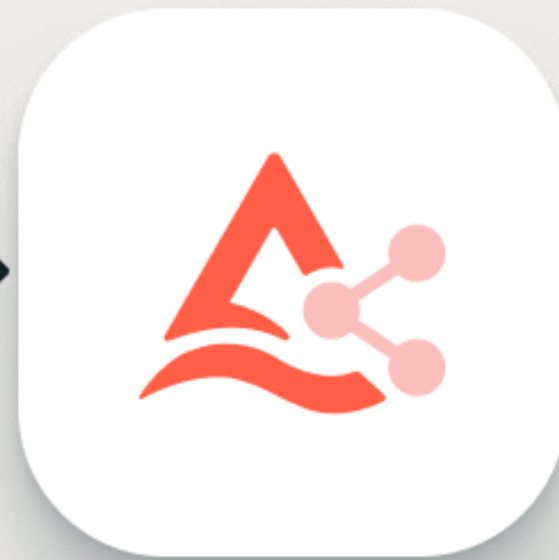
Iceberg?

**UC Iceberg
Tables**

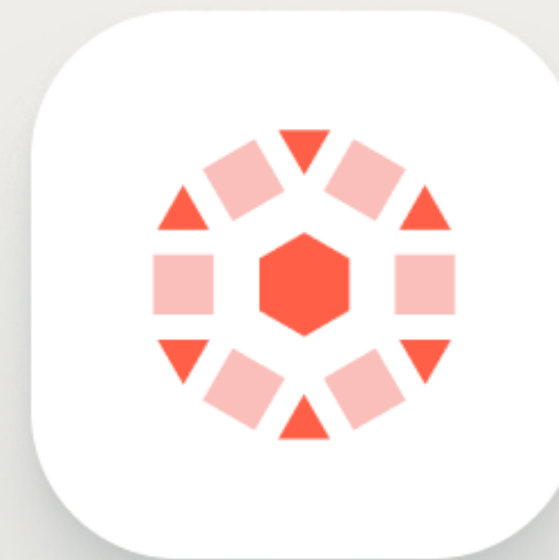


Delta Sharing

**Delta Sharing
Ecosystem**

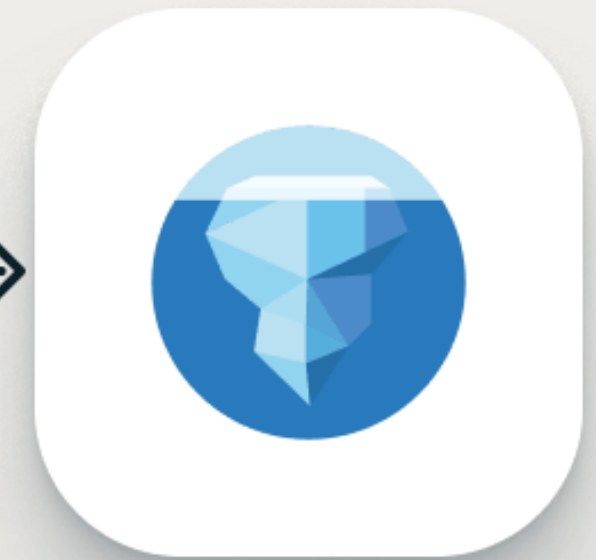


**All
UC Tables**



Delta Sharing

**Iceberg
IRC Clients**

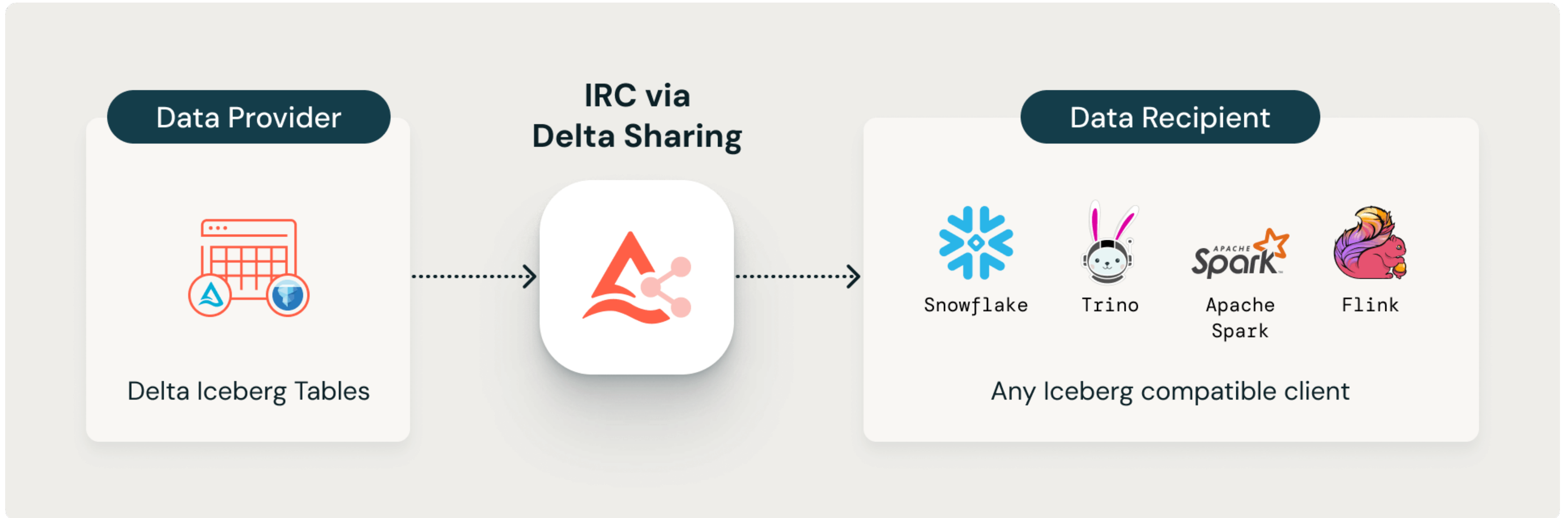


Share foreign and managed Iceberg tables
as Uniform to Delta Sharing clients
ecosystem without ETL

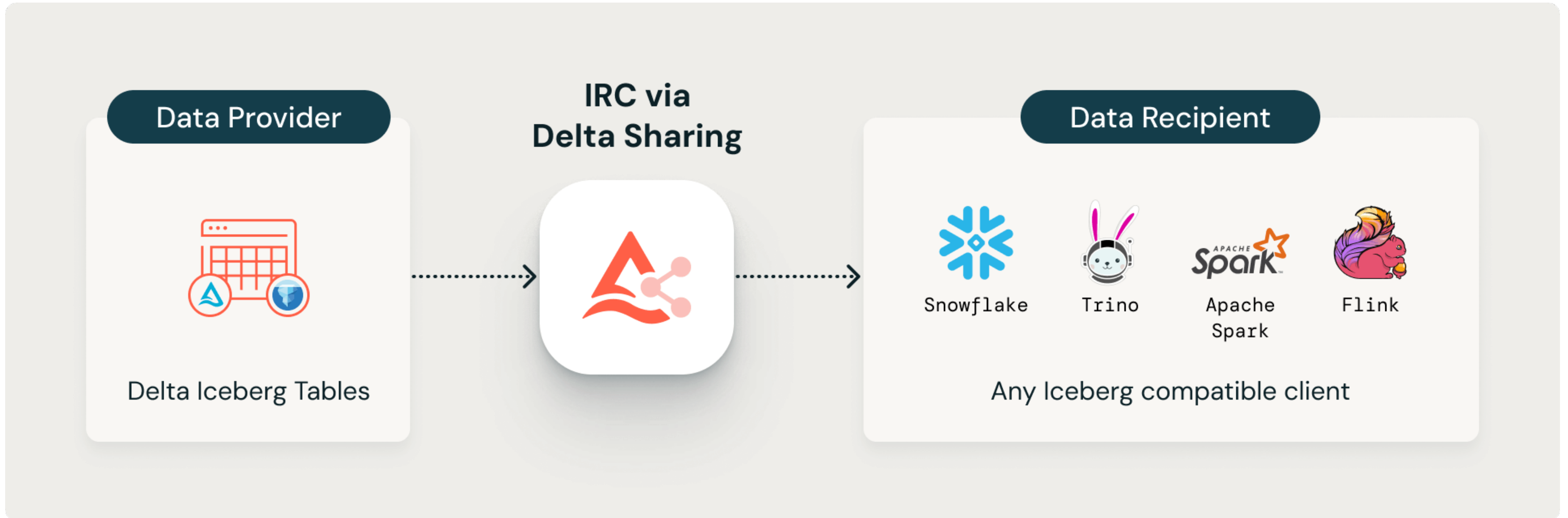
Expose Uniform-enabled Delta
tables through Delta Sharing to
Iceberg clients



Iceberg?



Iceberg?



Q&A



